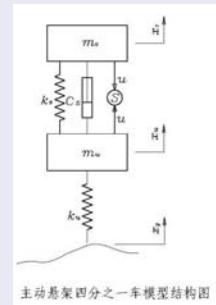


为什么可以仿真 (相似性)?

悬架的物理模型

悬架系统



主动悬架四分之一车模型结构图

悬架的数学模型

$$\ddot{x}_s = \frac{1}{m_s} u - \frac{k_s}{m_s} (x_s - x_u) - \frac{c_s}{m_s} (\dot{x}_s - \dot{x}_u)$$
$$\ddot{x}_u = \frac{1}{m_u} u \frac{k_s}{m_u} (x_s - x_u) + \frac{c_s}{m_u} (\dot{x}_s - \dot{x}_u) - \frac{k_u}{m_u} (x_u - x_g)$$

系统

- 通过各组成部分间相互作用而存在, 并具有某种特性或者功能
- **整体性是系统的主要特征:** 系统的特性不可能由内部的某个原器件或者子部分而实现, 系统的特性存在于系统各部分的相互作用中

模型

模型: 刻画系统的数学方程或者物理装置 (本课特指数学方程)

系统与模型

- 模型是对系统的刻画
- 刻画系统的模型不是唯一的, 也不是模型对系统的刻画越精确越好 (根据设计需要)
- 同一模型可能对应不同的系统: RLC 电路和机械位移系统

Matlab (mathematic laboratory)

基于矩阵运算的, 用于数学和工程计算的系统

- Matlab 处理的所有变量都是矩阵
- Matlab 具有一组范围广泛的获得图形输出的程序

Matlab 和各种工具箱一同使用

- 工具箱是一组成为 M 文件的专用文件
- 控制系统工具箱 — 控制系统的分析和设计

进入和退出Matlab

调用在线帮助工具

- **help 函数名**: 所列特定函数的目的和用法
- **help help**: 如何使用 help 函数

Navigation icons: back, forward, search, etc.

矩阵运算与逻辑运算

运算名	表达式	运算名	表达式
加	$a + b$	减	$a - b$
乘	$a * b$	乘方	$a \wedge 2$
矩阵转置	a'	矩阵相除	a/b

Table: Matrix operators

运算名	表达式	运算名	表达式
和	$\&$	或	$ $
非	$\sim$		

Table: Logical operators

Navigation icons: back, forward, search, etc.

逻辑运算

运算名	表达式	运算名	表达式
小于	<	小于或者等于	<=
大于	>	大于或者等于	>=
等于	==	不等于	~=

Table: Relational operators

- 前四个关系运算符只对实部进行比较
- 后两个既比较实部也比较虚部
- = 用于赋值语句, == 用于关系语句

特殊符号

符号	符号解释
[]	构成向量和矩阵
()	表示算数表达式的优先级
,	隔离下标和函数参数
%	注释

→默认不执行

分号;的用法

- 如果某个语句的最后字符是分号, 该语句仍然执行, 但结果不予显示
- 在输入矩阵时, 分号用来显示矩阵的某行已经结束, 但矩阵的最后一行不用分号

冒号用于生成向量

- $t = 1:5$ 从1到5
 $t =$
1 2 3 4 5 从1到5
- $t = 1:0.5:3$
 $t =$
1.0 1.5 2.0 2.5 3.0
- $t = 5:-1:2$
 $t =$
5 4 3 2

将向量输入到 Matlab 的语句

- $x = [1 \ 2 \ 3 \ -4 \ -5]$
- $x = [1, 2, 3, -4, -5]$
- 输出为行向量

将矩阵输入到 Matlab 的语句

- $\begin{bmatrix} 1.2 & 10 & 15 \\ 3 & 5.5 & 2 \\ 4 & 6.8 & 7 \end{bmatrix}$
- 空格, 个数, 回车
- $A = [1.2 \ 10 \ 15; 3 \ 5.5 \ 2; 4 \ 6.8 \ 7]$
 - $A = [1.2, 10, 15; 3, 5.5, 2; 4, 6.8, 7]$
 - $A = [1.2 \ 10 \ 15$ 回车键
3 5.5 2 回车键
4 6.8 7] 回车键

- $A(:,j)$ 是矩阵 A 的第 j 列
- $A(i,:)$ 是矩阵 A 的第 i 行
- $A(i,j)$ 是矩阵 A 的第 i 行和第 j 列的元素项

课堂练习

在Matlab中输入矩阵

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{pmatrix}$$

- $A([1,3,5],:)$ $A(:, [1,2,3,4,5])$
- (1) 提取矩阵全部奇数行, 所有列
 - (2) 提取矩阵第 3 2 1 行, 第 2 3 4 列构成的子矩阵
 - (3) 将 A 矩阵左右翻转, 即最后一列排在最前面
 - (4) 提取矩阵最后一个元素

课堂练习

在Matlab中输入矩阵

$$A = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{pmatrix}$$

- (1) 提取矩阵全部奇数行, 所有列
- (2) 提取矩阵第 3 2 1 行, 第 2 3 4 列构成的子矩阵
- (3) 将 A 矩阵左右翻转, 即最后一列排在最前面
- (4) 提取矩阵最后一个元素

- (1) $B_1 = A([1 \ 3 \ 5],:)$ 或者 $B_1 = A(1:2:end,:)$
- (2) $B_2 = A([3 \ 2 \ 1], [2 \ 3 \ 4])$
- (3) $B_3 = A(:, [5 \ 4 \ 3 \ 2 \ 1])$ 或者 $B_3 = A(:, end:-1:1)$
- (4) $B_4 = A(end,end)$

清除命令

- 清除工作站中存储的所有数据: `clear all`
不清理屏幕
- 清除工作站中变量 s 代表的数据: `clear s`
- 清除屏幕上显示的数据: `clc`
不清理工作站中存储的数据

`linspace` 生成从 n_1 到 n_2 的向量, 共 n 个点 (包括两个端点)

- 语句: $x = \text{linspace}(n_1, n_2, n)$ *linspace*
- 例: $x = \text{linspace}(-10, 10, 5)$
- 输出: $x = [-10 \ -5 \ 0 \ 5 \ 10]$

`logspace` 生成从 10^{n_1} 到 10^{n_2} , 按照对数间隔的向量, 共 n 个点 (包括两个端点)

- 语句: $x = \text{logspace}(n_1, n_2, n)$
- 例: $x = \text{logspace}(-1, 1, 10)$

$\sqrt{}$: `sqrt`

输入复数 $x = 1 + j\sqrt{3}$

复数可以借用函数 i 或者 j 输入

- 语句 1: $x = 1 + \text{sqrt}(3) * i$
- 语句 2: $x = 1 + \text{sqrt}(3) * j$
- 变量 i 和 j 在 Matlab 中已经被定义为单位虚数
- 可以定义新的变量代表单位虚数, 例如
 $ii = \text{sqrt}(-1)$, $x = 1 + \text{sqrt}(3) * ii$
- 如果想要回到预先设定的 $i = \sqrt{-1}$, 则需要输入 `clear i`

输入长度超过一行的长语句

- 语句通常用回车键或者 ; 表示语句结束
- 如果一个正在输入的语句过长, 无法在一行内输入, 就可以用三个 "." 构成的省略号 "..." 和回车键表示该语句在下一行继续
- 通过键盘录入: `disp(['abcdabcd...'`
- 换行继续录入: `abcd'])`
- 输出结果: `abcdabcdabcd`

display 展示

在一行内输入几个语句: 语句用 ";" 或者 "," 分隔

- `x = linspace(n1, n2, n), y = [4 3 5]`
- `x = linspace(n1, n2, n); y = [4 3 5]`

选择输出格式

- `format short; x`
- `format long; x`

输出格式的示例: `x = [1/3 0.00002]`

- `format short; x`
x
输出: 0.3333 0.0000
- `format long; x`
x
输出: 0.33333333333333 0.00002000000000

- Matlab 中所有的计算都按照双精度进行
- 语句一旦调用, 所选的格式就一直生效, 直到下次改变为止
- 如果尚未定义, 自动显示选择 `format short`
- 如果矩阵或者向量中的所有元素都是精确整数, 则 `format short` 和 `format long` 产生一样的结果

公用矩阵

函数	解释
<code>eye(n)</code>	产生 $n \times n$ 维单位矩阵
<code>ones(n)</code>	产生一个元素为 1 的 $n \times n$ 维矩阵
<code>ones(m,n)</code>	产生一个元素为 1 的 $m \times n$ 维矩阵
<code>zeros(n)</code>	产生一个元素为 0 的 $n \times n$ 维矩阵
<code>zeros(m,n)</code>	产生一个元素为 0 的 $m \times n$ 维矩阵

对角矩阵

x 是一个向量, `diag(x)` 产生一个对角线元素为 x 的对角矩阵

- `diag([ones(1,5)])`
- `diag(1:5)`

A 是一个方阵, `diag(A)` 就是由 A 的对角元素构成的向量

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$

- `diag(A)` $\begin{pmatrix} 1 \\ 5 \\ 9 \end{pmatrix}$ % 提取矩阵 A 的对角线元素
- `diag(diag(A))` $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 9 \end{bmatrix}$

语句 `diag(0,n)` 产生一个元素全为零的 $(n+1) \times (n+1)$ 维矩阵

- `diag(0,4)` % 产生 5 行 5 列的 0 矩阵

Matlab 中的运算

加减运算

- 矩阵维数相同, 否则提示错误

$$A = \begin{bmatrix} 2 & 4 & 6 \\ 3 & 5 & 7 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 3 & 4 \end{bmatrix}, \quad A - B = \begin{bmatrix} 1 & 2 & 6 \\ 3 & 2 & 3 \end{bmatrix}$$

- $x = \begin{bmatrix} 5 \\ 4 \\ 6 \end{bmatrix}$, 若输入 $y = x - 1$

$$y = \begin{bmatrix} 4 \\ 3 \\ 5 \end{bmatrix} \quad \% \text{ 所有的元素减去 1}$$

- $A - 1 = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}$

Navigation icons: back, forward, search, etc.

矩阵相乘

- 矩阵匹配相同, 否则提示错误

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad y = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}, \quad A = \begin{bmatrix} 1 & 1 & 2 \\ 3 & 4 & 0 \\ 1 & 2 & 5 \end{bmatrix}$$

- $x' * y, \quad x * y', \quad A * x, \quad A * 5, \quad 5 * A$

复数的表示

- 1 $z = a + bj$
- 2 相角 $\theta = \text{angle}(z)$
- 3 幅值 $r = \text{abs}(z)$
- 4 欧拉公式:
 $z = r * \exp(i * \theta)$

若 A 为复数矩阵

- 1 函数 $\text{angle}(A)$ 给出一个矩阵, 它由 A 的每一个元素的相角构成, 角度在 $(-\pi, \pi)$ 之间
- 2 函数 $\text{abs}(A)$ 给出一个矩阵, 它由 A 的每一个元素的绝对值构成

Navigation icons: back, forward, search, etc.

数组相乘 $z=x.*y$

- 两个数组中对应元素的乘积构成的同维数数组
 $x = [1 \ 2 \ 3], \quad y = [4 \ 5 \ 6] \quad z = [4 \ 10 \ 18]$
- $x.^2$ 给出了每个元素的平方构成的向量
- 如果矩阵 A 和矩阵 B 具有相同的维数, $A.*B$ 表示一个数组, 其元素就是 A 和 B 相应的元素的乘积

数组相除

- ① $x./y$ x 中对应元素除以 y 中的对应元素构成的数组
 $x.\backslash y$ y 中对应元素除以 x 中的对应元素构成的数组
- ② $A./B$ A 中对应元素除以 B 中的对应元素构成的数组
 $A.\backslash B$ B 中对应元素除以 A 中的对应元素构成的数组

若出现被 0 除的情况, Matlab 发出警告

- ① $\frac{0}{0}$ Nan
 $\frac{5}{0}$ Inf

仿真：从后台走到前台，从丑小鸭蜕变为天鹅

仿真

- 设计和测试的辅助手段：机械、电路和控制器

数字孪生（平行仿真）

- 虚拟世界和物理世界的互动：矿山，码头，战场
 - 虚拟世界：感知，决策
 - 物理世界：执行，创造使用价值

元宇宙

- 人们娱乐、生活乃至工作的虚拟时空，核心是数字创造、数字资产、数字交易、数字货币和数字消费
- 在用户体验方面，达到了真假难辨、虚实混同的境界
 - 流浪地球中的“数字生命计划”

控制系统数字仿真

于树友

Jilin University

shuyou@jlu.edu.cn

March 20, 2024

已学习的命令/语句

- `help`
- `x=logspace( $d_1, d_2, n$ )`
- `x=linspace( $n_1, n_2, n$ )` 注：共 n 个点
- 输入长度超过一行的长语句 注： $\dots$ ，但只和 `disp` 联合使用有效
- 选择输出格式
 - `format short, x`
 - `format long, x`
- 公用矩阵
 - `eye(n)` 单位矩阵
 - `ones(n)` 产生一个元素为一的 $n \times n$ 维矩阵
 - `ones(n,m)` 产生一个元素为一的 $n \times m$ 维矩阵
 - `zeros(n)` 产生一个元素为零的 $n \times n$ 维矩阵
 - `zeros(n,m)` 产生一个元素为零的 $n \times m$ 维矩阵

已学习的命令/语句

- `diag(ones(1,5))`
- `diag(A)`
- `diag(diag(A))`

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

$$\text{diag}(A) = \begin{pmatrix} 1 \\ 5 \\ 9 \end{pmatrix}$$

$$\text{diag}(\text{diag}(A)) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 9 \end{pmatrix}$$

`diag(0,n)` 产生一个元素全为零的 $(n+1) \times (n+1)$ 维矩阵

type 将函数的全部内容展开

已学习的命令/语句

矩阵运算

+, -, ×, ÷, 转置: `a'`, 乘方: `a^2`, `inv(a)`

关系运算符

>, >=, <, <=, ==, ~=

逻辑运算符

& ~ |
与 或 非

已学习的命令/语句

- : 的作用

`t=1:5` 例5

`t=1:2:15` 间隔2

`A(:,j)`

`A(:)` 返回列向量

`A([1 2 3],:)`

- Matlab中输入矩阵或向量

- 输入复数

`i^2=-1`

`j^2=-1`

`x=3+2*j`

Matlab 中的变量

使用变量前不需要定义变量的维数, 变量的维数在该变量第一次使用时会自动产生, 并且可以在今后改变

`A = [1 2 3; 4 5 6; 7 8 9]`

`x = [3 4 5]`

获得工作站空间中的变量清单: who/whos

- `>> who` 回车

Your variables are

A, x

- 若输入 `whos`, 屏幕上会显示当前工作空间中的所有变量信息

时间 & 日期

- `>> clock`

- `>> date`

- `>> cputime`

在 Matlab 中输入注释

符号 `%` 表示本行其余部分是注释, 应当忽略.

§ 1.3 计算矩阵函数

特征方程

方阵 A 的特征方程的根和 A 的特征值相同

A 的特征方程: $P = \text{poly}(A)$ 程序:

eg: $A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -6 & -11 & -6 \end{pmatrix}$ $A = [0 \ 1 \ 0; 0 \ 0 \ 1; -6 \ -11 \ -6];$
 $P = \text{poly}(A)$
 $P =$
1.0000 6.0000 11.0000 6.0000

矩的特殊多项式 (系数)

在这里多项式表示成行向量, 它包含按照降幂排列的多项式系数

故此例中多项式为: $P(s) = s^3 + 6s^2 + 11s + 6$

§ 1.3 计算矩阵函数

- 特征方程 $P=0$ 的根可以通过 $r = \text{roots}(P)$ 来求得
- eg: $q = \text{poly}(r)$ 可以将特征方程的根重新集合返回原来的多项式.

$\gg r = [-3 \ -2 \ -1]$

$\gg q = \text{poly}(r)$

$q =$

1 6 11 6

- 对于确定系数的传递函数, 劳斯稳定判据不再需要

§ 1.3 计算矩阵函数

- 多项式相加或相减

(1) 如果两个多项式具有相同的次数, 就可以将描述它们的系数的数组相加

(2) 如果两个多项式具有不同的次数(n 和 m , $n < m$), 就要在低次多项式的系数数组的左侧添 $m-n$ 个零。

```
>> a = [ 3 10 25 36 50 ];  
>> b = [ 0 0 1 2 10 ];  
>> a+b  
ans =  
    3    10    26    38    60
```

Navigation icons: back, forward, search, etc.

§ 1.3 计算矩阵函数

- 特征值和特征向量 $\text{eig}(A)$, $[x,D]=\text{eig}(A)$

(1) 如果 A 是 $n \times n$ 维矩阵, 那么满足 $Ax = \lambda x$ 的 n 个数 λ 就是 A 的特征值

(2) 如果 A 是实对称矩阵, 则其特征值是实数; 如果 A 不对称, 其特征值通常是复数。

(3) $\text{eig}(A)$ 生成由 A 的特征值构成的列向量; $[x,D]=\text{eig}(A)$ 生成 A 的特征值和特征向量。

其中对角矩阵 D 的对角元素是 A 的特征值; x 的列是对应的特征向量, 即 $Ax=xD$

Navigation icons: back, forward, search, etc.

§ 1.3 计算矩阵函数

- 多项式的乘积 (卷积) $c=\text{conv}(a,b)$

```
>> a = [ 3 10 25 36 50 ];  
>> b = [ 1 2 10 ];  
>> c=conv(a,b)
```

- (去卷积)多项式相除 $[q,r]=\text{deconv}(a,b)$

```
>> [q,r]=deconv(a,b)  
q=  
    3    4   -13           商  
r=  
    0    0    0   22  180      余数
```

$$3s^4 + 10s^3 + 25s^2 + 36s + 50 = (s^2 + 2s + 10)(3s^2 + 4s - 13) + 22s + 180$$

Navigation icons: back, forward, search, etc.

§ 1.3 计算矩阵函数

- 多项式求值: polyval, polyvalm

(1) 如果 P 是一个向量, 其元素是按降幂排列的多项式系数, 则 $\text{polyval}(P,s)$ 就是该多项式在 s 点的取值

```
>> P = [ 3 2 1 ]; % P(s)=3s^2 + 2s + 1  
>> polyval(P,5) % 多项式在s=5时的取值
```

(2) 如果 x 是一个多项式, 其系数为 P ; 则 $\text{polyvalm}(P,A)$

```
>> x = [ 2 3 1 ]; % x(s)=2s^2 + 3s + 1  
>> h=polyvalm(x, [ 2 0  
                  0 2 ])
```

$$h = 2 * \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}^2 + 3 * \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

注意: h 是 2×2 维矩阵

Navigation icons: back, forward, search, etc.

§ 1.3 计算矩阵函数

- 矩阵指数：命令 `expm(A)` 给出 $n \times n$ 维矩阵 A 的矩阵指数，即

$$\expm(A) = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \dots$$

注：某些函数名称后面加一个 "m"，超越函数就被解释为矩阵函数

- 方阵的逆 `inv(A)`

如果 A 不可逆，提示

Warning: Matrix is singular to working space.

§ 1.4 Matlab 程序设计语言基础

Matlab 语言的变量与常量

- Matlab 语言变量名应该由一个字母引导，后面可以跟数字，字母，下划线等。

`MYvar12`, `MY_var12`, `MYvar12_`

- Matlab 语言变量名区分大小写

§ 1.4 Matlab程序设计语言基础

Matlab 中的常量

eps	机器中浮点运算误差限，默认值为 2.2204×10^{-16} 若某个值的绝对值小于 eps，则可以认为这个量为 0
i 和 j	纯虚数， $i^2 = j^2 = -1$
Inf	无穷大，也可以写成 inf 即使遇到了以 0 为除数的运算也不会终止程序的运行， 而只给出一个“除 0”的警告，并将结果赋成 Inf
NaN	不定式 (not a number)，通常由 $\frac{0}{0}$ 运算， $\frac{inf}{inf}$ 运算得出
pi	圆周率 π 的双精度浮点。
lasterr	存放最新一次的错误信息。此变量为字符串型，如果在本次执行中没有出现过错误，则此变量为空字符串
lastwarn	存放最新一次的警告信息。如果在本次执行中没有出现过错误，则此变量为空字符串

§ 1.4 Matlab程序设计语言基础

数据结构

① 数值型数据

`format long, a;` `format short, a`

② 符号型：可以用于公式推导和数学问题的解析解法

将变量申明为符号变量

`syms` 变量名 变量类型

注释：

- 可以同时申明多个变量，中间用空格分隔，而不是用逗号等分隔；
- 变量类型：real, positive, negative
- 符号型数值可以通过变精度算法函数 `vpa` 以任意指定精度显示出来；
`vpa(A)` 或者 `vpa(A,n)`
其中 A 为需要显示的数值或矩阵，n 为指定的有效数字位数，前者以默认的十进制位数显示结果

§ 1.4 Matlab程序设计语言基础

③ Matlab的基本语句结构

直接赋值语句

函数调用语句

[返回变量]=函数名(输入变量列表)

eg: $\sin(2)$

④ 冒号表达式与子矩阵提取

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

§1.4 Matlab程序设计语言基础

基本数学运算

- 矩阵的代数运算
- 矩阵的逻辑运算
- 矩阵的比较运算

选择常考

- ① $C=A>B$ % 若 A 和 B 矩阵满足 $a_{ij} > b_{ij}$ 时, $c_{ij} = 1$; 否则, $c_{ij} = 0$
- ② find 函数可以查询出满足其关系的数组下标

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 0 \end{bmatrix}$$

- ③ $\text{find}(A \geq 5)$ 函数相当于先将 A 矩阵按列构成列向量, 然后再判断哪些元素大于或等于 5, 返回其下标。

$$[i,j]=\text{find}(A \geq 5)$$

$\text{find}(A==6)$ 返回 8

$\text{find}(A \geq 5)$ 返回 3 5 6 8

§1.4 Matlab程序设计语言基础

- `find(isnan(A))` 查出变量 A 中为 NaN 的各元素下标

$[i,j] = \text{find}(\text{isnan}(A))$

$h = \text{find}(\text{isnan}(A))$

- `all(A >= 5)`: 当矩阵的某列元素全大于等于 5 时, 相应元素为 1, 否则为 0。

`any(A >= 5)`: 当矩阵某列中含有元素大于等于 5 时, 相应元素为 1, 否则为 0。

课上小习题 2.1

如何编写程序, 判断一个矩阵 A 的元素均大于等于 5

§1.4 Matlab程序设计语言基础

- `find(isnan(A))` 查出变量 A 中为 NaN 的各元素下标

$[i,j] = \text{find}(\text{isnan}(A))$

$h = \text{find}(\text{isnan}(A))$

- `all(A >= 5)`: 当矩阵的某列元素全大于等于 5 时, 相应元素为 1, 否则为 0。

`any(A >= 5)`: 当矩阵某列中含有元素大于等于 5 时, 相应元素为 1, 否则为 0。

课上小习题 2.1

如何编写程序, 判断一个矩阵 A 的元素均大于等于 5

`all(A(:) >= 5)`

变量替换函数

$f_1 = \text{subs}(f, x_1, x_1^*)$ % 单个变量替换

$f_1 = \text{subs}(f, \{x_1, x_2, x_3, \dots, x_n\}, \{x_1^*, x_2^*, x_3^*, \dots, x_n^*\})$ % 多个变量替换

eg: 由表达式 $s = \frac{z+1}{z-1}$ 替换多项式中的 s 算子

```
syms z s;
```

```
p=
```

```
subs(p,s, $\frac{z+1}{z-1}$ ) % 变量替换并化简
```

控制系统数字仿真

于树友

Jilin University
shuyou@jlu.edu.cn

April 1, 2024

Navigation icons: back, forward, search, etc.

于树友 (控制科学与工程系)

Matlab for Control Engineers

April 1, 2024

1 / 28

复习

③ 特征值和特征向量 $Ax = \lambda x$

$$\lambda = \text{eig}(A)$$

$$[x, D] = \text{eig}(A)$$

- 对角矩阵 D 的对角元素是 A 的特征值;
- x 的列是相应特征值对应的特征向量.

Navigation icons: back, forward, search, etc.

于树友 (控制科学与工程系)

Matlab for Control Engineers

April 1, 2024

2 / 28

复习

- ④ 表征多项式的列向量相加, 减.

次数不同, 在低次多项式系数数组的左侧添加 $n-m$ 个零.

- ⑤ 多项式的乘积 $c = \text{conv}(a, b)$

多项式去卷积 (除)

$$[q, r] = \text{deconv}(a, b)$$

q 商, r 余数, a 被除数, b 除数 $a = bq + r$

多项式求值

$\text{polyval}(P, 5)$

$\text{polyvalm}(P, A)$

复习

- ⑥ 矩阵指数 $\text{expm}(A)$ e^A

- ⑦ 矩阵逆 $\text{inv}(A)$

- ⑧ Matlab 中的常量

eps 误差限, 默认值为 2.2204×10^{-16} 。

i 和 j 虚数

Inf 无穷大

NaN 不定式, 由 $\frac{0}{0}$ 运算, $\frac{\text{inf}}{\text{inf}}$ 运算得出。

pi 圆周率 π 的双精度浮点。

lasterr 存放最新一次的错误信息。

lastwarn 存放最新一次的警告信息。

9 符号型变量的定义

`syms` 变量名 □ 变量名 类型

变量类型: `real` `positive` `negative`

10 矩阵的比较运算

`C=A>B` $a_{ij} > b_{ij}$ 时, $c_{ij} = 1$; 否则, $c_{ij} = 0$

填空

`find(isnan(A))` 查出变量 `A` 中为 NaN 的各元素下标

`find(A>=5)` 查出满足 $a_{ij} \geq 5$ 的数组下标, 把矩阵视为列向量

`[i,j]=find(A>=5)`

`all(A>5)` 矩阵的某列元素全大于 5 时, 相应元素为 1, 否则为 0.

`any(A>5)` 当矩阵某列中含有的元素大于 5 时, 相应元素为 1, 否则为 0.

解析结果的化简与变换

变量替换函数

$f_1 = \text{subs}(f, x_1, x_1^*)$ 用 x_1^* 替换 f 中的 x_1 % 单个变量替换

$f_1 = \text{subs}(f, \{x_1, x_2, x_3, \dots, x_n\}, \{x_1^*, x_2^*, x_3^*, \dots, x_n^*\})$ % 多个变量替换

eg: 由表达式 $s = \frac{z+1}{z-1}$ 替换多项式中的 s 算子

`syms a b phi`

`syms z s;`

$r = \sqrt{a^2 + b^2 + \phi}$

`p=`

`subs(p, s,  $\frac{z+1}{z-1}$ )` % 变量替换

$f = \text{subs}(r, a, b \times \phi)$

↓

$(b \times \phi)^2 + b^2 + \phi$

§2.3 Matlab语言的流程结构

- 循环语句结构
- 条件语句结构
- 开关语句结构

§2.3.1 循环结构

循环结构可以由 **for** 或者 **while** 语句引导, 用 **end** 语句结束, 在这两个语句之间的部分称为循环体.

实现: $s = \sum_{i=1}^n i$

- for i=v

循环体结构

end

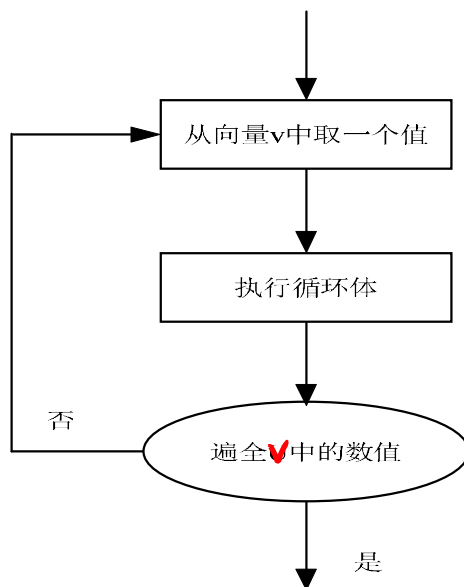
s=0

for i=1:5

s=i+s

end

§2.3.1 循环结构



- 在 for 循环结构中, v 为一个向量,
- 循环变量 i 每次从 v 向量取一个数值, 执行一次循环体的内容,
- 如此下去, 直至执行完 v 向量中所有分量, 将自动结束循环体的执行.

Navigation icons: back, forward, search, etc.

§2.3.1 循环结构

while 条件式

循环体结构

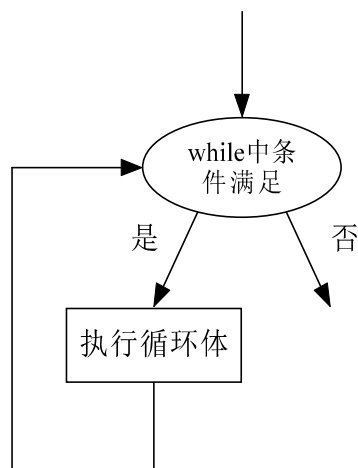
end

```
s=0; i=1;
```

```
while (i<= 5)
```

```
    s=s+i; i=i+1;
```

```
end
```



- **while** 循环中的 "条件式" 是一个逻辑表达式, 若其值为真则将自动执行循环体的结构;
- 执行完再判定 "条件式" 的真伪, 为真则仍然执行结构体, 否则退出循环结构

Navigation icons: back, forward, search, etc.

§2.3.1 循环结构

eg: 使用循环结构求解 $\sum_{i=1}^{100} i$

```
s=0
```

```
for i=1:100
```

```
    s=s+i
```

```
end
```

```
s=0; i=1;
```

```
while i <= 100
```

```
    s=s+i;
```

```
    i=i+1;
```

```
end
```

- 循环语句在 Matlab 里可以嵌套使用:
可在 for 下使用 while, 或者相反使用;
- 在循环结构中如果使用 break 语句, 则可以结束上一层的循环结构.

跳出当前循环

§2.3.1 循环结构

sum 函数: 向量 x 的各元素求和

如果 A 为矩阵, sum(A) 返回一个行向量, 每一个元素是列和.

上述循环语句可以用 sum(1:100) 来求得.

例: 求解级数求和问题 $\sum_{i=1}^{10000}$

§2.3.1 循环结构

sum 函数：向量 x 的各元素求和

如果 A 为矩阵, $\text{sum}(A)$ 返回一个行向量, 每一个元素是列和.
上述循环语句可以用 $\text{sum}(1:100)$ 来求得.

例：求解级数求和问题 $\sum_{i=1}^{10000} \left(\frac{1}{2^i} + \frac{1}{3^i} \right)$

```
s=0;
```

```
for i=1:10000
```

```
    s=s+ $\frac{1}{2^i}$  +  $\frac{1}{3^i}$ 
```

```
end;
```

```
toc
```

Navigation icons: back, forward, search, etc.

§2.3.1 循环结构

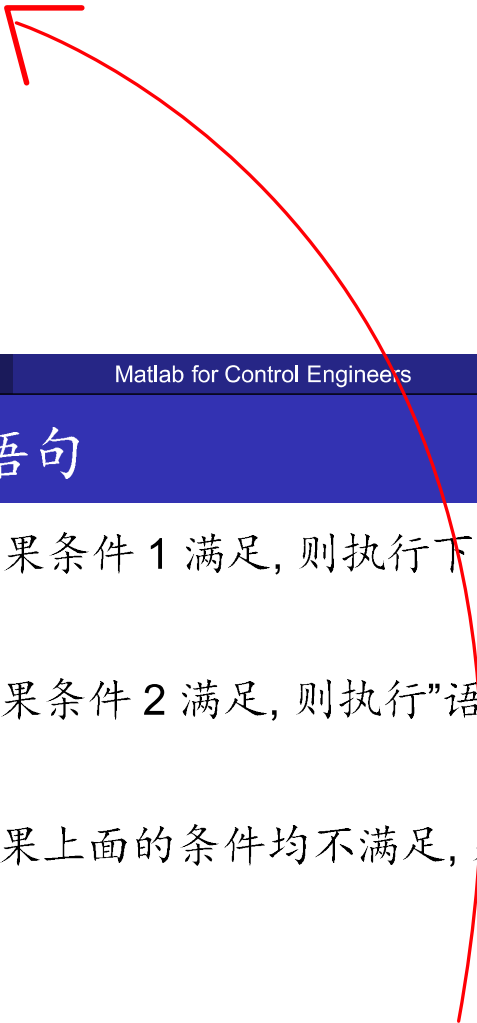
例：求出满足 $\sum_{i=1}^m i < 10000$ 的最小 m 值

Navigation icons: back, forward, search, etc.

§2.3.1 循环结构

例：求出满足 $\sum_{i=1}^m i < 10000$ 的最小 m 值

```
s=0;
m=0;
while (s<=10000)      % 可以没有"()"
    m=m+1;
    s=s+m;
end
s = s-m;
m = m-1;
```



Navigation icons: back, forward, search, etc.

§2.3.2 条件结构语句

```
if (条件 1)          % 如果条件 1 满足, 则执行下面的段落
    语句组 1
elseif (条件 2)      % 如果条件 2 满足, 则执行"语句组 2"
    语句组 2
else                  % 如果上面的条件均不满足, 则执行"语句组 3"
    语句组 3
end
```

取绝对值的运算

```
if x>=0;
    h=x;
else
    h=-x
end
```

```
>> s=0;m=0
for i=1:10000
    m=m+1;s=s+m;
    if s>10000
        break
    end
end
[s - m, m - 1]
```

Navigation icons: back, forward, search, etc.

§2.3.3 开关结构

```
switch 开关表达式
case 表达式 1
    语句段 1
case 表达式 2, ..., 表达式 n
    语句段 2
otherwise
    语句段 n
end
```

Navigation icons: back, forward, search, etc.

§2.3.3 开关结构

```
switch 开关表达式
case 表达式 1
    语句段 1
case 表达式 2, ..., 表达式 n
    语句段 2
otherwise
    语句段 n
end
```

求绝对值

```
a=-5;
switch a >= 0
case 1
    b=a
case 0
    b=-a
end
```

Navigation icons: back, forward, search, etc.

§2.3.3 开关结构

switch 开关表达式

case 表达式 1

语句段 1

case {表达式 2, ..., 表达式 n}

语句段 2

otherwise

语句段 n

end

a=-5;

switch a >= 0

case 1

b=a

case 0

b=-a

end

开关语句的关键是对“开关表达式”值的判断。

当开关表达式的值等于某个 **case** 语句后面的条件时, 程序将转移到该组语句中执行, 执行完成后程序转出开关体继续向下执行。

§2.3.3 开关结构

注意:

- switch 后面跟的语句可以是一个判断式, 或者任意的命令;
- case 后面则是说明命令可能出现的执行结果, 在 Matlab 中直接判断输入表达式的返回结果非 0 即 1.
- 当需要在开关表达式满足若干表达式之一时执行某一程序段, 则应该把这样的表达式用大括号括起来, 中间用逗号分隔.
- 这样的结构是 Matlab 语言定义的单元结构.
- 当前面枚举的各个表达式均不满足时, 则将执行 otherwise 语句后面的语句段.

§2.3.4 试探结构

try

语句段 1

catch me

语句段 2

end

- 本语句结构首先试探性地执行语句段 1,
- 如果在此执行过程中出现错误, 则将错误信息自动地赋给保留的 `lasterr` 变量, 并终止这段语句的执行,
- 转而执行语句段 2 中的语句.

举例:

- 不保险但速度快的算法放在 **try** 段落中,
- 保险的程序放在 **catch** 段落中,
- 原问题的求解更加可靠, 且能使程序高速执行.

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

例一：生成数列

生成一个数列, 并求该数列的前 $n \geq 3$ 项和, 数列满足

$$a(1) = 1, a(2) = 1, a(i) = a(i-1) + a(i-2).$$

◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡ ↺ 🔍 ↻

例一：生成数列

生成一个数列, 并求该数列的前 $n \geq 3$ 项和, 数列满足

$$a(1) = 1, a(2) = 1, a(i) = a(i-1) + a(i-2).$$

程序：

```
a(1)=1; a(2)=1;  
s=2;  
for i=3:n  
    a(i)=a(i-1)+a(i-2);  
    s=s+a(i)  
end  
a
```

例二：n 维数组从大到小排序 $A = \begin{bmatrix} 3 & 7 & 16 & 25 & 38 \end{bmatrix}$

例二: n 维数组从大到小排序 $A = [3 \ 7 \ 16 \ 25 \ 38]$

```
for i=1:n
    for j=(i+1):n
        if A(i)>A(j)
            A(i)=A(i);
            A(j)=A(j);
        else
            hs=A(i);
            A(i)=A(j);
            A(j)=hs;
        end
    end
end
end
```

Navigation icons: back, forward, search, etc.

例二: n 维数组从大到小排序 $A = [3 \ 7 \ 16 \ 25 \ 38]$

```
for i=1:n
    for j=(i+1):n
        if A(i)>A(j)
            A(i)=A(i);
            A(j)=A(j);
        else
            hs=A(i);
            A(i)=A(j);
            A(j)=hs;
        end
    end
end
end
```

无用

```
for i=1:n
    for j=(i+1):n
        if A(i)<A(j)
            hs=A(i);
            A(i)=A(j);
            A(j)=hs;
        end
    end
end
end
```

Navigation icons: back, forward, search, etc.

- M 脚本文件

Matlab 中的 M 文件只能对 Matlab 工作空间中的数据进行处理, 文件中的所有语句的执行结果也完全返回到工作空间.

- M 函数文件

.m 文件

Matlab 函数编写与调试

- Matlab 语言函数的基本结构

`function [返回变量列表]=函数名(输入变量列表)`

- 结构说明

返回变量列表: 由逗号或者空格来分隔

输入变量列表: 用逗号来分隔

- 特点

(1) 注释说明语句段, 由 % 引导, 键入 `help` 命令显示出来

(2) 输入、返回变量格式的检测

(3) 函数体语句

Matlab 编程要领

- 编程时如果输入返回变量格式不正确, 则应给出相关提示.
- Matlab 函数是一个变量处理单元, 它从主调函数接受变量, 对之进行处理后, 将结果返回到主调函数中.
除输入和输出变量外, 其他在函数内部产生的所有变量都是局部变量, 在函数调用结束后, 这些变量将消失.
- 函数名与文件名相同, 否则提示错误信息.

Matlab 函数编写与调试

```
function k=myfun(n)
if nargin ~= 1, error('输入变量个数错误'); end
if nargout > 1, error('输出变量个数过多'); end
if abs(n - floor(n)) > eps | n < 0
    error('n 应该为非负整数')
end
if n > 1
    k=n*myfun(n-1)
elseif
    any([0 1])==n
    k=1
end
```

Handwritten notes:

- 输入变量个数* (Input variable count) - points to `nargin`
- abs(n) 取绝对值* (abs(n) takes absolute value) - points to `abs(n)`
- 判断是否为整数* (Judge if integer) - points to `abs(n - floor(n)) > eps`
- lin → 0* - points to `n < 0`
- 变量* (Variable) - points to `n`
- 阶乘* (Factorial) - points to `k=n*myfun(n-1)`
- 0!=1!=1 为已知, 为本函数出口* (0!=1!=1 is known, exit point of this function) - points to `any([0 1])==n`
- 向下取整* (Floor) - points to `floor(n)`

注: **floor:** rounds the elements of x to the nearest integers
error: 中止程序, 并提示出错信息

可输入个数处理

- ① 利用 Matlab 函数可以递归调用的特性建立无限个输入函数调用格式

卷积

```
function a=convsv(varargin)
    a=1
    for i=1:length(varargin)
        a=conv(a,varargin(i))
    end
```

变量维数

% conv 表示向量相乘
% 输入自动视为一个向量
% 其中 i 代表下标为 i 的输入元

Matlab 函数编写与调试

程序流程控制

- input 指令: 指示用户从键盘输入数值、字符串和表达式, 并接受该输入

```
a=input('please input a number')
```

- pause 指令: 程序暂停运行, 等待用户按任意键后继续该命令

```
pause % 按任意键继续
```

```
pause(n) % 在继续执行前暂停 n 秒钟
```

- break 指令: 跳出当下循环体

1. 用程序实现 A 的转置

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

1. 用程序实现 A 的转置

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \\ 16 & 17 & 18 & 19 & 20 \\ 21 & 22 & 23 & 24 & 25 \end{bmatrix}$$

```
for i=1:5;
    for j=1:5;
        B(j,i)=A(i,j);
    end
end
B
```

课堂练习：统计考试成绩

若成绩小于 60 分, 输出 Failed; 若成绩介于 [60,80), 输出 Pass; 若成绩大于 80 分, 输出 Pass with excellent.

课堂练习：统计考试成绩

若成绩小于 60 分, 输出 Failed; 若成绩介于 [60,80), 输出 Pass; 若成绩大于 80 分, 输出 Pass with excellent.

程序：

```
n=input('请输入成绩')
```

```
k=floor(n/10)
```

```
switch k
```

```
case{8, 9, 10}
```

```
    disp('Pass with excellent')
```

```
case{6, 7}
```

```
    disp('Pass')
```

```
otherwise
```

```
    disp('Failed')
```

```
end
```

课堂练习：统计考试成绩

若成绩小于 60 分, 输出 Failed; 若成绩介于 [60,80), 输出 Pass; 若成绩大于 80 分, 输出 Pass with excellent.

课堂练习：统计考试成绩

若成绩小于 60 分, 输出 Failed; 若成绩介于 [60,80), 输出 Pass; 若成绩大于 80 分, 输出 Pass with excellent.

程序：

```
grade=78;
```

```
if grade > 60 & grade < 80
```

```
    disp('Pass')
```

```
elseif grade >= 80
```

```
    disp('Pass with excellent')
```

```
else
```

```
    disp('Failed')
```

```
end
```

控制系统数字仿真

于 树友

Jilin University
shuyou@jlu.edu.cn

April 3, 2024

Matlab 函数编写与调试

- M 脚本文件

Matlab 中的 M 文件只能对 Matlab 工作空间中的数据进行处理, 文件中的所有语句的执行结果也完全返回到工作空间。

- M 函数文件

.m 文件

Matlab 函数编写与调试

- Matlab 语言函数的基本结构

`function [返回变量列表]=函数名(输入变量列表)`

- 结构说明

返回变量列表: 由逗号或者空格来分隔

输入变量列表: 用逗号来分隔

- 特点

- (1) 注释说明语句段, 由 % 引导, 键入 help 命令显示出来
- (2) 输入、返回变量格式的检测
- (3) 函数体语句

Matlab 函数编写与调试

Matlab 编程要领

- 编程时如果输入返回变量格式不正确, 则应给出相关提示.
- Matlab 函数是一个变量处理单元, 它从主调函数接受变量, 对之进行处理后, 将结果返回到主调函数中.
除输入和输出变量外, 其他在函数内部产生的所有变量都是局部变量, 在函数调用结束后, 这些变量将消失.
- 函数名与文件名相同, 否则提示错误信息.

Matlab 函数编写与调试

```
function k=myfun(n)
if nargin ~= 1, error('输入变量个数错误'); end
if nargin > 1, error('输出变量个数过多'); end
if abs(n - floor(n)) > eps | n < 0
    error('n 应该为非负整数')
end
if n > 1                % 若 n > 1 进行递归调用
    k=n*myfun(n-1)
elseif
    any([0 1])==n        % 0!=1!=1 为已知, 为本函数出口
    k=1
end
```

注: floor: rounds the elements of x to the nearest integers
error: 中止程序, 并提示出错信息

Navigation icons: back, forward, search, etc.

Matlab 函数编写与调试

可输入个数处理

- ① 如何建立无限个输入函数调用格式
- ② 利用 Matlab 函数可以递归调用的特性

```
function a=convs(varargin)    % conv 表示向量相乘
    a=1                      % 输入自动视为一个向量
    for i=1:length(varargin)
        a=conv(a,varargin(i)) % 其中 i 代表下标为 i 的输入元
    end
```

Navigation icons: back, forward, search, etc.

Matlab 函数编写与调试

程序流程控制

- `input` 指令: 指示用户从键盘输入数值、字符串和表达式, 并接受该输入

```
a=input('please input a number')
```

- `pause` 指令: 程序暂停运行, 等待用户按任意键后继续该命令

```
pause % 按任意键继续
```

```
pause(n) % 在继续执行前暂停 n 秒钟
```

- `break` 指令: 跳出当下循环体

绘制响应曲线

命令 `plot` 能够生成线性 x-y 图

- `loglog`, `semilogx`, `semilogy` 能够绘制对数坐标图
所有这些命令的用法相同, 他们只影响坐标轴的标度数据的显示
- 如果 `x` 和 `y` 是长度相同的向量, `plot(x, y)` 就以 `(x, y)` 中的值来绘制曲线

绘制响应曲线

绘制多条曲线

- 方法一: `plot(x1, y1, x2, y2, x3, y3)`
- 方法二:

```
plot(x1, y1)
hold on    % 保留已有的图片
plot(x2, y2)
hold on
plot(x3, y3)
hold off
```

错误方法:

```
plot(x1, y1)
plot(x2, y2)
plot(x3, y3)
```

每一次画图清除已有图, 最后只剩下曲线 $(x3, y3)$ 对应的图

Navigation icons: back, forward, search, etc.

绘制响应曲线

- ③ 添加网格线、图形标题和 x 和 y 轴标记

```
grid    % 网格线
title   % 标题
xlabel  % x 轴标记
ylabel  % y 轴标记
```

- ④ 在图形上书写文字

- `text(x, y, 'string')`

将引号中的文字添加到当前坐标轴位置 (x, y) 上

eg: `text(3, 0.45, 'sin(t)')`

- `gtext('string')`

显示图形窗口, 画出一个十字细线, 等待用户鼠标点击或者按下任意键; 十字细线可以用鼠标定位, 按下鼠标左键或者任意键将该文本字符串写到图形所选位置.

Navigation icons: back, forward, search, etc.

绘制响应曲线

5 虚数或复数图像画法

如果 z 是复数向量, `plot(z)` 就相当于 `plot(real(z),imag(z))`

6 极坐标图

命令 `polar(theta, rho)` 会在极坐标中给出以角度 θ (单位为弧度) 和半径为 ρ 的点阵构成的图形

绘制响应曲线

手动坐标轴标度

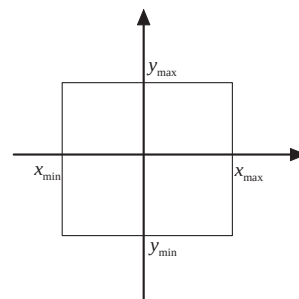
- 如果希望在由

$$v = [x_{min} \ x_{max} \ y_{min} \ y_{max}]$$

规定的范围内划线, 则可以输入命令 `axis(v)`, 其中 v 是一个 4 元向量.

- 这个命令将坐标轴标度设置成预先设置的界限.

注释: 对于对数图形, v 的元素个数是极大值和极小值的 $\log_{10}$ 值.



绘制响应曲线

- 执行命令 `axis(v)` 后, 当前坐标轴的标度会一直保持, 再次键入 `axis` 就可以恢复自动标注
- `axis('square')` 在屏幕上将图形区域设置为正方形
- `axis('normal')` 是长高比设置返回正常状态
- `axis('equal')` 设置的长高比使每个坐标轴上相等刻度的分度标记增量具有相等的间隔

绘制响应曲线

标记符号绘图

`plot(x, y, 'o')` 采用 “o” 标记符号绘制点图

注意:

- 使用该命令绘制的 “o” 的标记符号并没有用直线或者曲线连接
- 若需实线连接, 可使用命令 `plot(x,y,x,y,'o')` 分两次来绘制图像.

绘制响应曲线

可以使用的线型和点型

线型		点型	
实线	-	点	.
虚线	--	加号	+
点线	:	星号	*
点划线	-.	圆	o
		x 记号	x

绘制响应曲线

可以使用的颜色

语句: `plot(x,y,'r')` %采用红色绘制曲线
`plot(x,y,'+g')` %采用绿色 '+' 绘制曲线

颜色	
红色	r
绿色	g
蓝色	b
黑色	black

绘制响应曲线

生成希腊字母:

采用 ‘\字符名称’ 的方式:

字母	
α	\alpha
β	\beta
γ	\gamma

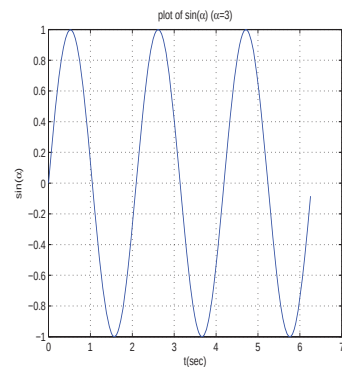
生成无穷符号、水平和垂直短箭头等

符号		符号	
∞	\infty	$\circ$	\circ
$\leftarrow$	\leftarrow	$\rightarrow$	\rightarrow
$\uparrow$	\uparrow	$\downarrow$	\downarrow

绘图示例程序

程序 1.1

```
>> t = 0 : 0.011 * pi : 2 * pi;  
>> alpha=3  
>> y = sin(alpha * t);  
>> plot(t,y)  
>> grid  
>> title('plot of sin(\alpha) (\alpha=3)')  
>> xlabel('t(sec)')  
>> ylabel('sin(\alpha)')
```

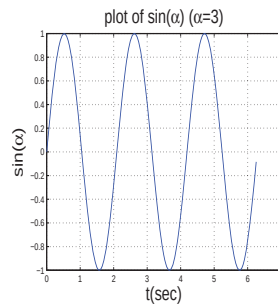


绘图示例程序

将程序 1.1 最后三行做如下改变：

字体尺寸

```
>> title('plot of sin(\alpha)(\alpha=3)','FontSize',20)
>> xlabel('t(sec)','FontSize',20)
>> ylabel('sin(\alpha)','FontSize',20)
```



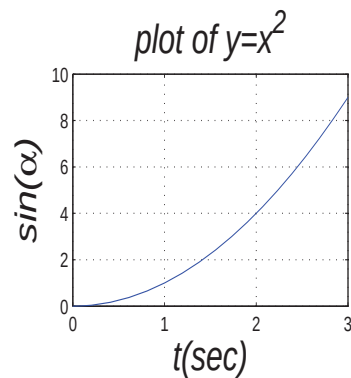
绘图示例程序

字体的相关设置：

- 'FontSize', 20 % 设置字体尺寸
- 'Fontangle', 'italic' % 设置字体倾斜
- 'Fontname', 'Times New Roman' % 设置字体名称
- 利用 “-” 生成下标, 例如: x_1 产生 x_1
- 利用 “^” 生成下标, 例如: x^2 产生 x^2

程序 1.3

```
>> x=0:0.1:3;  
>> y = x.^2;  
>> plot(x,y), grid  
>> title('plot of  $y=x^2$ ', 'FontSize', 20, 'Fontangle', 'italic')  
>> xlabel('t(sec)', 'FontSize',20, 'Fontangle', 'italic')  
>> ylabel('sin(\alpha)', 'FontSize',20, 'Fontangle', 'italic')
```



绘图示例程序

- 分区图形命令 (一张图中画出若干小图)

`subplot(m, n, p)`

- 图形显示被划分为 $m \times n$ 个较小的子窗口
- 这些子窗口从左到右, 由上到下标号
- 整数 p 指明曲线绘制的窗口

- 下面在 $0 \leq t \leq \pi$ 范围内绘制两条曲线:

$$y_1 = \sin(t)$$

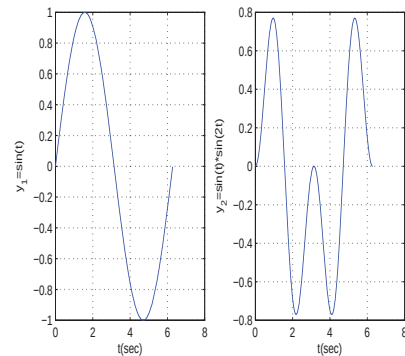
$$y_2 = \sin(t) * \sin(2t)$$

绘制两条曲线

程序1.4

```
>> t = 0 : 0.01 : 2 * pi;  
>> y1=sin(t);  
>> y2 = sin(t) .* sin(2 * t);  
>> subplot(1,2,1)  
>> plot(t,y1)  
>> grid  
>> xlabel('t(sec)')  
>> ylabel('y_1=sin(t)')  
>> subplot(1,2,2)  
>> plot(t,y2)  
>> grid  
>> xlabel('t(sec)')  
>> ylabel('y_2 = sin(t) * sin(2t)')
```

考试必有画图题



线性代数问题的 Matlab 求解

矩阵的参数化分析

① 矩阵的行列式(determinant)

det(A) % 支持符号函数运算

② 矩阵的迹 (trace): 矩阵对角线上各元素之和

trace(A)

③ 矩阵的秩 (rank)

rank(A)

④ 矩阵的"2 范数"

norm(A)

Lyapunov 方程求解

- Lyapunov 方程

$$AX + XA^T = -C$$

其中 A 和 C 为给定矩阵, 且 C 为对称正定矩阵

- Matlab 提供如下函数, 可以立即求解满足 Lyapunov 方程的矩阵 X

$$X = \text{lyap}(A, C)$$

控制系统数字仿真

于树友

Jilin University
shuyou@jlu.edu.cn

April 8, 2024

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

求解特点

Matlab 语言可以对一阶常微分方程组求取数值解; 其他类型的微分方程可以通过适合的算法转换成可解的一阶微分方程 (组) 求解.

- 用四阶五阶 Runge-Kutta 算法求解一阶常微分方程的函数 ode45
 $[t, x] = \text{ode45}(\text{'方程函数名'}, \text{tspan}, x_0, \text{选择项}, \text{附加参数})$
 - tspan: 数值解时的初始和终止时间
 - x_0 : 初始状态变量
 - 选择项: 求解微分方程的一些控制参数
 - 附加参数: 在求解函数和方程描述函数之间的传递
 - 一般采用变步长算法, 返回的 x 是一个矩阵, 其列数为 n (变量 x 的维数), 行数等于 t 的列数, 每一行对应于对应时间节点处的状态变量向量的转置. t 为仿真结果的自变量构成的向量.

注: x 的最后一行返回的是 t_p 时刻的系统输出值, 其中 $\text{tspan} = [t_0, t_p]$.

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

eg: 考虑微分方程组:

$$\begin{cases} \dot{x}_1(t) = -x_2(t) - x_3(t) \\ \dot{x}_2(t) = x_1(t) + ax_2(t) \\ \dot{x}_3(t) = b + [x_1(t) - c]x_3(t) \end{cases}$$

选定:

$$\begin{cases} a = b = 0.2 \\ c = 5.7 \end{cases}$$

且 $x_1(0) = x_2(0) = x_3(0) = 0$, 求解该微分方程组.

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

- 第一步: 若想求解这个微分方程, 需要用户自己去编写一个 Matlab 函数来描述这个常微分方程.

```
function dx=ressler(t, x)
    dx=[-x(2)-x(3);
        x(1)+0.2* x(2);
        0.2+(x(1)-5.7)*x(3)] % 对比此函数和给出的数学方程, 编写这样的函数是很直观的.
```

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

- 第二步: 求解微分方程的数值解

```
>> x0=[0; 0; 0] % 微分方程初值设定
```

```
>> [t, y]=ode45('ressler', [0,100], x0)
```

```
% 求解微分方程, 注意函数名要有 ' ' 号
```

```
>> plot(t, y) % 绘制各个状态的响应曲线
```

Navigation icons: back, forward, search, etc.

- 描写微分方程:

```
function dx=rossler(t, x, flag, a, b, c)
```

```
% 加入附加参数, flag 非常重要
```

```
dx=[-x(2)-x(3);
```

```
      x(1)+a*x(2);
```

```
      b+(x(1)-c)*x(3)];
```

- 求解微分方程组:

```
a=0.2; b=0.2; c=0.57;
```

```
[t, y] = ode45('rossler', [0, 100], x0, [], a, b, c)
```

Navigation icons: back, forward, search, etc.

- 求解如下的常微分方程：

$$\ddot{y} = 2\dot{y} + 3y + 1$$

$$x_1 = y$$

$$\dot{x}_1 = \dot{y} = x_2$$

$$x_2 = \dot{y}$$

$$\dot{x}_2 = \ddot{y} = x_3$$

$$x_3 = \ddot{y}$$

$$\dot{x}_3 = \ddot{\dot{y}} = 2x_3 + 3x_2 + x_1 + 1$$

- 求解如下的常微分方程：

$$\ddot{y} = 2\dot{y} + 3y + 1$$

- 将常微分方程转换成微分方程组：

$$x = [y \quad \dot{y} \quad \ddot{y}]^T$$

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = x_3$$

$$\dot{x}_3 = 2x_3 + 3x_2 + x_1 + 1$$

最优问题的 Matlab 求解

缺点: 容易陷入局部极小值

- 无约束最优问题

$$\min_x F(x)$$

- 求解该优化问题的 Matlab 函数

$$[x, f_{opt}, key, c] = fminsearch(@Fun, x_0, opt)$$

- Fun 为要求解问题的数学描述;
- x_0 为自变量的起始搜索点;
- opt 为最优化工具箱的选项设定;
- x 为返回的解;
- f_{opt} 是目标函数在 x 点的值;
- key 函数返回的条件
1—已经求解出方程的解
0—未搜索到方程的解
- c 为解的附加信息

常微分方程的 Matlab 求解

- 有约束的最优问题

$$\begin{aligned} \min_x & f(x) \\ \text{s.t.} & Ax \leq B \\ & A_{eq}x = B_{eq} \\ & x_m \leq x \leq x_M \quad \% \text{按元素给出} \\ & c(x) \leq 0 \\ & c_{eq}(x) = 0 \end{aligned}$$

- 求解该优化问题的 Matlab 函数

$$[x, f_{opt}, key, c] = fmincon(@Fun, x_0, A, B, Aeq, Beq, x_m, x_M, @CFun, opt)$$

注: 若未搜索到优化问题的解 (优化问题在搜索区间没有可行解), 则 key=0; 可以考虑改变初值, 或修改控制参数 opt, 再进行优化, 以得出期望的最优值.

常微分方程的 Matlab 求解

$$\begin{aligned} & \underset{x}{\text{minimize}} \quad e^{x_1} (4x_1^2 + 2x_2^2 + 4x_1x_2 + 2x_2 + 1) \\ & \text{subject to} \\ & x_1 + x_2 \leq 0 \\ & -x_1x_2 + x_1 + x_2 \geq 1.5 \\ & x_1x_2 \geq -10 \\ & -10 \leq x_1, x_2 \leq 10 \end{aligned}$$

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

目标函数:

```
function y=myobj(x)
```

$$y = \exp(x(1)) * (4 * x(1) \wedge 2 + 2 * x(2) \wedge 2 + 4 * x(1) * x(2) + 2 * x(2) + 1)$$

约束变量:

```
function [c,ce]=mycon(x)
```

```
ce=[]; % 无等式约束
```

```
c=[x(1) + x(2); x(1) * x(2) - x(1) - x(2) + 1.5;  
-10 - x(1) * x(2)]
```

注: $x(1)+x(2)$ 也可采用 $AX \leq B$ 来描述

Navigation icons: back, forward, search, etc.

常微分方程的 Matlab 求解

主函数:

```
A=[];      B=[];  
Aeq=[];    Beq=[];  
xm=[-10;-10]; xM=[10;10];  
x0=[5;5];
```

```
[x, fopt, key, c] = fmincon(@myobj, x0, A, B, Aeq, Beq, xm, xM, @mycon)
```

注意:

常微分方程的求解和优化问题的求解非常重要

- 控制是系统结构和参数的优化
- 系统或者对象是由常微分方程描述的

动态系统 Matlab 分析的初步研究

- 1 用 Matlab 进行部分分式展开

$$\frac{B(s)}{A(s)} = \frac{\text{num}}{\text{den}} = \frac{b_0 s^n + b_1 s^{n-1} + \dots + b_n}{s^n + a_1 s^{n-1} + \dots + a_n}$$

其中某些 a_i 和 b_j 可以是零.

行向量 num 和 den 分别指明了传递函数的分子分母系数:

$$\text{num} = [b_0 \ b_1 \ \dots \ b_n]$$

$$\text{den} = [1 \ a_1 \ \dots \ a_n]$$

命令 $[r, p, k] = \text{residue}(\text{num}, \text{den})$ 可以求出两个多项式 $B(s)$ 和 $A(s)$ 之比的部分分式展开式的留数 r , 极点 p 和直接项 k . 即:

$$\frac{B(s)}{A(s)} = \frac{r(1)}{s - p(1)} + \frac{r(2)}{s - p(2)} + \dots + \frac{r(n)}{s - p(n)} + k(s)$$

动态系统 Matlab 分析的初步研究

- 例题: 已知传递函数

$$\frac{B(s)}{A(s)} = \frac{2s^3 + 5s^2 + 3s + 6}{s^3 + 6s^2 + 11s + 6}$$

对其进行部分分式分解.

```
>> num = [ 2 5 3 6 ]
```

```
>> den = [ 1 6 11 6 ]
```

```
>> [r, p, k] = residue(num, den)
```

输出结果: $r = \begin{bmatrix} -6 \\ -4 \\ 3 \end{bmatrix}$ $p = \begin{bmatrix} -3 \\ -2 \\ -1 \end{bmatrix}$ $k = 2$

即:

$$\frac{B(s)}{A(s)} = \frac{-6}{s+3} + \frac{-4}{s+2} + \frac{3}{s+1} + 2$$

Navigation icons: back, forward, search, etc.

动态系统 Matlab 分析的初步研究

- 命令 residue 还能根据部分分式展开构成(分子和分母) 多项式, 即:

$$[num, den] = residue(r, p, k)$$

程序:

```
>> r = [ -6 -4 -3 ];
```

```
>> p = [ -3 -2 -1 ];
```

```
>> k=2;
```

```
>> [num, den] = residue(r, p, k)
```

程序输出:

```
>> num = [ 2 5 3 6 ];
```

```
>> den = [ 1 6 11 6 ]; % 降幂排列多项式系数
```

Navigation icons: back, forward, search, etc.

动态系统 Matlab 分析的初步研究

- 为了获得 s 的多项式的比显示, 可以输入如下 Matlab 程序:

```
[num, den] = residue(r, p, k)
printsys(num, den, 's')
```

输出:

$$\frac{num}{den} = \frac{2s^3 + 5s^2 + 3s + 6}{s^3 + 6s^2 + 11s + 6}$$

- 注意:** 如果 $p(j) = p(j+1) = \dots = p(j+m-1)$, 则 $p(j)$ 是 m 重根, 该情况下展开式应包括如下的形式:

$$\frac{r(j)}{s - p(j)} + \frac{r(j+1)}{[s - p(j)]^2} + \dots + \frac{r(j+m-1)}{[s - p(j)]^m}$$

Navigation icons: back, forward, search, etc.

动态系统 Matlab 分析的初步研究

- 例题: 用 Matlab 将

$$\frac{B(s)}{A(s)} = \frac{s^2+2s+3}{(s+1)^3} = \frac{s^2+2s+3}{s^3+3s^2+3s+1}$$

展开为部分分式形式.

程序:

```
>> num = [ 1 2 3 ];
>> den = [ 1 3 3 1 ];
>> [r, p, k] = residue(num, den)
```

程序输出:

$$r = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}; \quad p = \begin{bmatrix} -1 \\ -1 \\ -1 \end{bmatrix}; \quad k = []$$

$$\text{即: } \frac{B(s)}{A(s)} = \frac{1}{s+1} + \frac{0}{(s+1)^2} + \frac{2}{(s+1)^3}$$

Navigation icons: back, forward, search, etc.

动态系统 Matlab 分析的初步研究

② 用 Matlab 求 $B(s)/A(s)$ 的零点和极点

(1) $[z, p, k] = tf2zp(num, den)$ 可以求得 $\frac{B(s)}{A(s)}$ 的零极点增益
eg:

$$\frac{B(s)}{A(s)} = \frac{4s^2 + 16s + 12}{s^4 + 12s^3 + 44s^2 + 48s}$$

程序:

```
>> num = [ 4 16 12 ];  
>> den = [ 1 12 44 48 0 ];  
>> [z, p, k] = tf2zp(num, den)
```

输出结果:

$$z = \begin{bmatrix} -3 \\ -1 \end{bmatrix}; \quad p = \begin{bmatrix} 0 \\ -6 \\ -4 \\ -2 \end{bmatrix}; \quad k=4;$$

注: (a) 不能直接用 `printsys`
(b) 与 `residue` 进行区分

$$\text{即: } \frac{B(s)}{A(s)} = \frac{4(s+3)(s+1)}{s(s+6)(s+4)(s+2)}$$

动态系统 Matlab 分析的初步研究

(2) 如果零极点和增益已经给定, 求 num/den

程序:

```
>> z=[-1;-3]; % 输入为列向量  
>> p=[0;-2;-4;-6];  
>> k=4;  
>> [num, den] = zp2tf(z, p, k)  
>> printsys(num, den, 's')
```

函数 `zp2tf` 将零极点模型转化为传递函数模型

习题 1

- 求 $F(s) = \frac{s^5+8s^4+23s^3+35s^2+28s+3}{s^3+6s^2+8s}$ 的拉普拉斯逆变换

Navigation icons: back, forward, search, etc.

习题 1

- 求 $F(s) = \frac{s^5+8s^4+23s^3+35s^2+28s+3}{s^3+6s^2+8s}$ 的拉普拉斯逆变换

程序:

```
>> num=[1 8 23 35 28 3];  
>> den=[1 6 8];  
>> [r, p, k] = residue(num, den)
```

输出结果:

```
r = [ 0.3750  
      0.2500  
      0.3750 ]  
p = [ -4  
      -2  
       0 ]  
k = [ 1 2 3 ]
```

- 因此 $F(s) = (s^2 + 2s + 3) + \frac{0.375}{s+4} + \frac{0.25}{s+2} + \frac{0.375}{s}$

$F(s)$ 的拉普拉斯逆变换是:

$$f(t) = \frac{d^2}{dt^2} \delta(t) + 2 \frac{d}{dt} \delta(t) + 3 \delta(t) + 0.375 e^{-4t} + 0.25 e^{-2t} + 0.375$$

$(t > 0)$

Navigation icons: back, forward, search, etc.

习题 2

- 给定 $B(s)/A(s)$ 的零极点 and 增益, 试用 Matlab 来获得函数的 $B(s)/A(s)$

- (1) 没有零点, 极点位于 $-1+2j$ 和 $-1-2j$, $k=10$;
- (2) 一个零点为零, 极点位于 $-1+2j$ 和 $-1-2j$, $k=10$;
- (3) 一个零点为 -1 , 极点位于 $-2, -4, -8$, $k=12$

课堂习题 2

- 程序1:

```
z=[ ];  
p=[-1+2*j;-1-2*j];  
k=10;  
[num,den]=zp2tf(z,p,k);  
printsys(num,den)
```

输出:

$$\frac{10}{s^2 + 2s + 5}$$

课堂习题 2

- 程序2:

```
z=[0];  
p = [-1 + 2 * j; -1 - 2 * j];  
k=10;  
[num, den] = zp2tf(z, p, k);  
printsys(num, den)
```

输出:

$$\frac{10s}{s^2 + 2s + 5}$$

课堂习题 2

- 程序3:

```
z=[-1];  
p=[-2; -4; -8];  
k=12;  
[num, den] = zp2tf(z, p, k);  
printsys(num, den)
```

输出:

$$\frac{12s + 12}{s^3 + 14s^2 + 56s + 64}$$

§ 2.2 动态系统数学模型的转换

- 传递函数模型到零点-极点模型的变换
- 零点-极点模型到传递函数模型的变换
- 传递函数模型到状态空间模型的变换
- 状态空间模型到传递函数模型的变换

§ 2.2 动态系统数学模型的转换

1. 传递函数模型到状态空间模型的变换

命令 $[A, B, C, D] = \text{tf2ss}(\text{num}, \text{den})$

将具有传递函数形式

$$\frac{Y(s)}{U(s)} = \frac{\text{num}}{\text{den}} = C(SI - A)^{-1}B + D$$

的系统变换为状态空间形式

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

注: 任何系统的状态空间表达式都不唯一, Matlab 命令只给出这些表达式中的一种可能形式.

§ 2.2 动态系统数学模型的转换

eg: $\frac{Y(s)}{U(s)} = \frac{10s+10}{s^3+6s^2+5s+10}$

程序输出:

程序:

```
num=[10 10];
```

```
den=[1 6 5 10];
```

```
[A,B,C,D]= tf2ss (num, den)
```

$$A = \begin{pmatrix} -6 & -5 & -10 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, B = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

$$c=[0 \ 10 \ 10], \ D=[0]$$

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

2. 状态空间模型到传递函数模型的变换

传递函数模型通常只用于描述单输入-单输出系统.

(1) 对于单输出, 输入量多于 1 的系统必须指定是第几个输入对输出的传递函数.

(2) 若只有一个输入, 则无需指定

```
[num, den]=ss2tf(A, B, C, D, iu)
```

```
[num, den]=ss2tf(A, B, C, D)
```

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

eg: 已知状态空间方程

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -5.008 & -25.1026 & -5.03247 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} 0 \\ 25.04 \\ -121.005 \end{pmatrix} u$$
$$y = [1 \ 0 \ 0] \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

程序:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -5.008 & -25.1026 & -5.03247 \end{pmatrix}; B = \begin{pmatrix} 0 \\ 25.04 \\ -121.005 \end{pmatrix}; C = [1 \ 0 \ 0];$$
$$D = 0$$

$$[\text{num}, \text{den}] = \text{ss2tf}(A, B, C, D) \quad \text{或者} \quad [\text{num}, \text{den}] = \text{ss2tf}(A, B, C, D, 1)$$

§2.2 动态系统数学模型的转换

考虑系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -25 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

这个系统包含 2 个输入, 2 个输出, 共 4 个传递函数. 在考虑 u_1 时, 假设输入 $u_2=0$; 反之亦然.

§2.2 动态系统数学模型的转换

考虑系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -25 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

这个系统包含 2 个输入, 2 个输出, 共 4 个传递函数. 在考虑 u_1 时, 假设输入 $u_2=0$; 反之亦然.

程序:

$$A = \begin{pmatrix} 0 & 1 \\ -25 & -4 \end{pmatrix}; B = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}; C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}; D = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

[num, den]=ss2tf(A, B, C, D,1)

[num, den]=ss2tf(A, B, C, D, 2)

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

4 个传递函数分别为:

$$\frac{Y_1(s)}{U_1(s)} = \frac{s+4}{s^2+4s+25}$$

$$\frac{Y_1(s)}{U_2(s)} = \frac{s+5}{s^2+4s+25}$$

$$\frac{Y_2(s)}{U_1(s)} = \frac{-25}{s^2+4s+25}$$

$$\frac{Y_2(s)}{U_2(s)} = \frac{s-25}{s^2+4s+25}$$

Navigation icons: back, forward, search, etc.

控制系统数字仿真

于树友

Jilin University
shuyou@jlu.edu.cn

April 15, 2024

Navigation icons: back, forward, search, etc.

学习过的命令/函数

① 求解一阶常微分方程组

$[t,x]=ode45('函数名', tspan, x_0, 选项, 附加参数)$

Navigation icons: back, forward, search, etc.

学习过的命令/函数

- ❶ 求解一阶常微分方程组

`[t,x]=ode45('函数名', tspan, x0, 选项, 附加参数)`

- ❷ 求解无约束优化问题

$$\min_x f(x)$$
$$[x, fopt, key, c] = fminsearch('fun', x_0, opt).$$

x: 最优解对应的自变量

fopt: 最优解

key=1: 最优解已找到

key=0: 最优解未找到

c: 附加信息

Navigation icons: back, forward, search, etc.

学习过的命令/函数

- ❸ 求解约束最优化问题

$$\min_x f(x)$$

s.t. $Ax \leq B$

约束函数

$$A_{eq}x \leq B_{eq}$$

function [c ce] = mycon(x)

$$x_m \leq x \leq x_M$$

c: 不等式约束

$$c(x) \leq 0$$

ce: 等式约束

$$c_{eq}(x) = 0$$

`[x,fopt,key,c] =`

`fmincon('函数名', x0, A, B, Aeq, Beq, xm, xM, '约束函数名', opt)`

Navigation icons: back, forward, search, etc.

学习过的命令/函数

- 用 Matlab 进行部分分式展开

```
[r,p,k]=residue(num, den)    % 多重极点的处理
```

```
[num,den]=residue(r, p, k)
```

Navigation icons: back, forward, search, etc.

学习过的命令/函数:

用 Matlab 求零点和极点:

```
[z, p, k]=tf2zp(num, den)
```

如果零极点已给定, 求传递函数:

```
[num, den]=zp2tf(z, p, k)
```

注意: tf2zp 和 residue 的不同.

前者:

$$G(s) = \frac{k \prod (s+b_i)}{\prod (s+a_i)}$$

后者:

$$G(s) = \sum \frac{b_i}{s+a_i} + k$$

Navigation icons: back, forward, search, etc.

学习过的命令/函数:

动态系统数学模型的转换

- 传递函数模型到零点-极点模型的变换
tf2zp
- 零点-极点模型到传递函数模型的变换
zp2tf
- 传递函数模型到状态空间模型的变换
tf2ss
- 状态空间模型到传递函数模型的变换
ss2tf

§ 2.2 动态系统数学模型的转换

1. 传递函数模型到状态空间模型的变换

命令 `[A, B, C, D] = tf2ss (num, den)`

将具有传递函数形式

$$\frac{Y(s)}{U(s)} = \frac{num}{den} = C(SI - A)^{-1}B + D$$

的系统变换为状态空间形式

$$\dot{x} = Ax + Bu$$

$$y = Cx + Du$$

注: 任何系统的状态空间表达式都不唯一, Matlab 命令只给出这些表达式中的一种可能形式.

§ 2.2 动态系统数学模型的转换

eg: $\frac{Y(s)}{U(s)} = \frac{10s+10}{s^3+6s^2+5s+10}$

程序:

```
num=[10 10];
```

```
den=[1 6 5 10];
```

```
[A,B,C,D]= tf2ss (num, den)
```

程序输出:

$$A = \begin{pmatrix} -6 & -5 & -10 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, B = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$$

```
c=[0 10 10], D=[0]
```

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

2. 状态空间模型到传递函数模型的变换

传递函数模型通常只用于描述单输入-单输出系统.

(1) 对于单输出, 输入量多于 1 的系统必须指定是第几个输入对输出的传递函数.

(2) 若只有一个输入, 则无需指定

```
[num, den]=ss2tf(A, B, C, D, iu)
```

```
[num, den]=ss2tf(A, B, C, D)
```

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

eg: 已知状态空间方程

$$\begin{pmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -5.008 & -25.1026 & -5.03247 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} + \begin{pmatrix} 0 \\ 25.04 \\ -121.005 \end{pmatrix} u$$

$$y = [1 \ 0 \ 0] \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

程序:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -5.008 & -25.1026 & -5.03247 \end{pmatrix}; B = \begin{pmatrix} 0 \\ 25.04 \\ -121.005 \end{pmatrix}; C = [1 \ 0 \ 0]; D = 0$$

$$[\text{num}, \text{den}] = \text{ss2tf}(A, B, C, D) \quad \text{或者} \quad [\text{num}, \text{den}] = \text{ss2tf}(A, B, C, D, 1)$$

Navigation icons: back, forward, search, etc.

§2.2 动态系统数学模型的转换

考虑系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -25 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

这个系统包含 2 个输入, 2 个输出, 共 4 个传递函数. 在考虑 u_1 时, 假设输入 $u_2=0$; 反之亦然.

Navigation icons: back, forward, search, etc.

§2.2 动态系统数学模型的转换

考虑系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -25 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$
$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix}$$

这个系统包含 2 个输入, 2 个输出, 共 4 个传递函数. 在考虑 u_1 时, 假设输入 $u_2=0$; 反之亦然.

程序:

$$A = \begin{pmatrix} 0 & 1 \\ -25 & -4 \end{pmatrix}; B = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}; C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}; D = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

[num, den]=ss2tf(A, B, C, D,1)

[num, den]=ss2tf(A, B, C, D, 2)

Navigation icons: back, forward, search, etc.

§ 2.2 动态系统数学模型的转换

4 个传递函数分别为:

$$\frac{Y_1(s)}{U_1(s)} = \frac{s+4}{s^2+4s+25}$$

$$\frac{Y_1(s)}{U_2(s)} = \frac{s+5}{s^2+4s+25}$$

$$\frac{Y_2(s)}{U_1(s)} = \frac{-25}{s^2+4s+25}$$

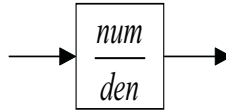
$$\frac{Y_2(s)}{U_2(s)} = \frac{s-25}{s^2+4s+25}$$

Navigation icons: back, forward, search, etc.

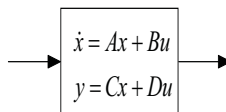
§ 2.3 框图形式系统的 Matlab 表达方式

模型仅用于描述、认知系统。系统将用于各种变换、操作。

系统的传递函数描述: $\text{sys}=\text{tf}(\text{num},\text{den})$

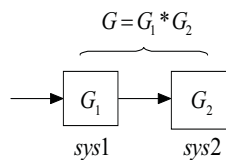


系统的状态空间描述: $\text{sys}=\text{ss}(\text{A},\text{B},\text{C},\text{D})$



§ 2.3 框图形式系统的 Matlab 表达方式

(1) 串连连接的框图化简



$\text{sys1}=\text{tf}(\text{num1}, \text{den1})$

或: $\text{sys1}=\text{ss}(\text{A1}, \text{B1}, \text{C1}, \text{D1})$

$\text{sys2}=\text{tf}(\text{num2},\text{den2})$

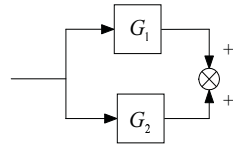
$\text{sys2}=\text{ss}(\text{A2}, \text{B2}, \text{C2}, \text{D2})$

$\text{sy}=\text{series}(\text{sys1}, \text{sys2})$

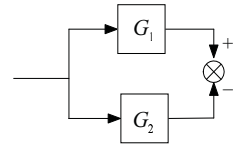
$\text{sy}=\text{series}(\text{sys1}, \text{sys2})$

§ 2.3 框图形式系统的 Matlab 表达方式

(2) 并连接框图化简



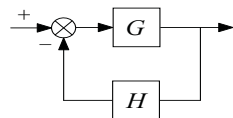
$$sys = parallel(sys1, sys2)$$



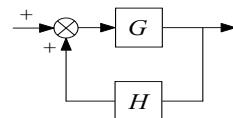
$$sys = parallel(sys1, -sys2)$$

§ 2.3 框图形式系统的 Matlab 表达方式

反馈连接的框图化简:



负反馈系统



正反馈系统

如果 G 和 H 采用传递函数形式来定义:

$$sysg = tf(numg, deng)$$

$$sysh = tf(numh, denh)$$

- 整个反馈系统 (负反馈) 可以表示为:

$$sys = feedback(sysg, sysh)$$

或者

$$[num, den] = feedback(sysg, sysh)$$

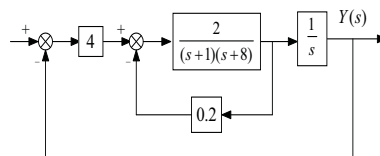
§ 2.3 框图形式系统的 Matlab 表达方式

- 如果该系统具有单位反馈函数, 则有 $H=1$
 $\text{sys}=\text{feedback}(\text{sysg},1)$ 或者 $\text{sys}=\text{feedback}(\text{sysg},1)$
- 如果系统包含正反馈 (Matlab 假设反馈是负反馈)
 $\text{sys}=\text{feedback}(\text{sysg},-\text{sysh})$

Navigation icons: back, forward, search, etc.

§2.3 框图形式系统的 Matlab 表达方式

eg: 试用 Matlab 求闭环传递函数



程序:

```
>> sysg1=[4];  
>> numg2=[2];deng2=[1 9 8];sysg2=tf(numg2,deng2);  
>> numg3=[1];deng3=[1 0];sysg3=tf(numg3,deng3);  
>> sysh=[0.2];  
>> sys2=feedback(sysg2,sysh);  
>> sys1=series(sysg1,sys2);  
>> sys2=series(sys1,sysg3);  
>> sys=feedback(sys2,[1]);  
>> sys_ss = ss(sys);
```

Navigation icons: back, forward, search, etc.

§ 2.3 框图形式系统的 Matlab 表达方式

- 为了获得某个框图的传递函数的状态空间表达式, 分子的次数必须低于或者等于分母的次数.
- 为了获取相应的传递函数, 可以采用 `sys_tf=tf(sys_ss)`

§2.3 框图形式系统的 Matlab 表达方式

零, 极点对消 minreal

eg: 考虑传递函数 $\frac{Y(s)}{U(s)} = \frac{(s+1)(s+2)}{(s+1)(s+3)(s+4)} = \frac{s^2+3s+2}{s^3+8s^2+19s+12}$

程序: num=[1 3 2];

```
den=[1 8 19 12];
```

```
sys=tf(num,den)
```

```
sys_min=minreal(sys)
```

注:设计系统时不建议采用零、极点对消。

尤其是当存在不稳定的零、极点对消发生时.

第三章: 瞬态相应分析

本章介绍当输入为时域输入时获得系统响应的 Matlab 方法.

§ 3.2 阶跃响应

如果已知 num 和 den (传递函数的分子和分母), 就可以用

`sys=tf(num,den);` `sys=ss(A,B,C,D)`

来 定义该系统.

采用

`step(sys)` 或是直接用 `step(num, den)`, `step(A, B, C, D)`

来产生单位阶跃响应的图形, 并 在屏幕上显示一条响应曲线.

§ 3.2 阶跃响应

% 计算的时间间隔 Δt 和响应时间的长度由 Matlab 确定.

% 如果希望 Matlab 每隔 Δt 秒计算一次响应值, 而且画出 $0 \leq t \leq T$ 的响应曲线, 就可以在程序中输入语句

$$t=0 : \Delta t : T$$

然后采用命令

step(sys, t) 或是 step(num, den, t)

§ 3.2 阶跃响应

% 如果 step 命令具有左端参数, 例如

`y=step(sys, t)` 或者 `y=step(num,den,t)`

和

`[y,t]=step(sys, t)` 或者 `[y, t]=step(num, den, t)`

那么 Matlab 会产生该系统的阶跃响应,但是不在屏幕上显示图形

§ 3.2 阶跃响应

eg: 已知机械系统如右图.

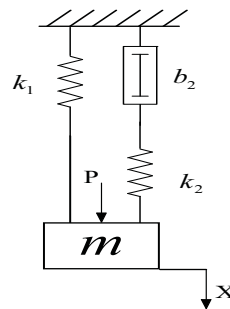
其传递函数为

$$\frac{X(s)}{P(s)} = \frac{b_2 S + k_2}{mb_2 S^3 + mb_2 S^2 + (k_1 + k_2)b_2 S + k_1 k_2}$$

假设 $m=0.1\text{kg}$, $b_2 = 0.4\text{N.s/m}$,

$k_1 = 6\text{N/m}$, $k_2 = 4\text{N/m}$. $P(t)$ 为

幅值为 10 的阶跃力, 试求响应 $x(t)$.



§ 3.2 阶跃响应

eg: 已知机械系统如右图.

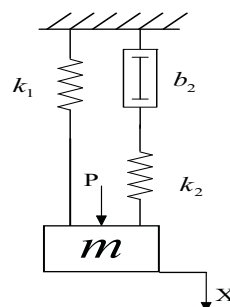
其传递函数为

$$\frac{X(s)}{P(s)} = \frac{b_2 S + k_2}{mb_2 S^3 + mb_2 S^2 + (k_1 + k_2)b_2 S + k_1 k_2}$$

假设 $m=0.1\text{kg}$, $b_2 = 0.4\text{N.s/m}$,

$k_1 = 6\text{N/m}$, $k_2 = 4\text{N/m}$. $P(t)$ 为

幅值为 10 的阶跃力, 试求响应 $x(t)$.



$$\text{解: } \frac{X(s)}{P(s)} = \frac{0.4S+4}{0.04S^3+0.4S^2+4S+24} = \frac{10S+100}{S^3+10S^2+100S+600}$$

因为 step 是单位阶跃响应, 若想求 $P(t)=10\text{N}$ 时的响应 $x(t)$, 可以求

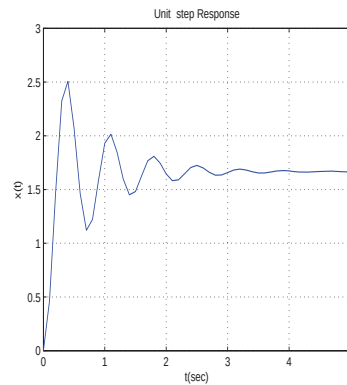
$$\frac{X(s)}{P(s)} = \frac{10 \cdot (10S+100)}{S^3+10S^2+100S+600} \text{ 的单位阶跃响应. (为什么可以这么做?)}$$

线性系统的叠加原理

§ 3.2 阶跃响应

程序:

```
>> t=0:0.5:5;  
>> num=[100 1000];  
>> den=[1 10 100 600];  
>> sys=tf(num,den);  
>> y=step(sys,t); > step(sys)  
>> plot(t,y)  
>> grid  
>> title('Unit-step Response')  
>> xlabel('t(sec)')  
>> ylabel('x(t)')
```



习题 1

(1) 用计算方法求如下高阶系统的单位阶跃响应;

$$\frac{C(s)}{R(s)} = \frac{3S^3 + 25S^2 + 72S + 80}{S^4 + 8S^3 + 40S^2 + 96S + 80}$$

并将图画在 $0 \leq x \leq 3, 0 \leq y \leq 1.2$ 的图形中.

(2) 当 $R(s)$ 是单位阶跃函数时, 用 Matlab 求 $C(s)$ 的部分分式展开式.

习题 1

(1) 用计算方法求如下高阶系统的单位阶跃响应;

$$\frac{C(s)}{R(s)} = \frac{3S^3 + 25S^2 + 72S + 80}{S^4 + 8S^3 + 40S^2 + 96S + 80}$$

并将图画在 $0 \leq x \leq 3, 0 \leq y \leq 1.2$ 的图形中.

(2) 当 $R(s)$ 是单位阶跃函数时, 用 Matlab 求 $C(s)$ 的部分分式展开式.

程序 1:

```
num=[3 25 72 80];
```

```
den=[1 8 40 96 80];
```

```
step(num,den);
```

画图:

```
v=[0 3 0 1.2];
```

```
axis(v);
```

```
grid
```

Navigation icons: back, forward, search, etc.

习题 1

(2) 当 $R(s)$ 是单位阶跃函数时, 系统响应的解析解

$$\Phi(s) = R(s) * \frac{C(s)}{R(s)} = \frac{1}{s} \frac{3S^3 + 25S^2 + 72S + 80}{S^4 + 8S^3 + 40S^2 + 96S + 80}$$

输出结果:

程序 2:

```
num1=[3 25 72 80];
```

```
den1=[1 8 40 96 80 0];
```

```
[r,p,k]=residue(num1,den1);
```

r=

```
-0.2812 - 0.1719i  
-0.2812 + 0.1719i  
-0.4375  
-0.3750  
1.0000
```

p=

```
-2.0000 + 4.0000i  
-2.0000 - 4.0000i  
-2.0000  
-2.0000  
0
```

k=

```
[];
```

Navigation icons: back, forward, search, etc.

控制系统数字仿真

于树友

Jilin University
shuyou@jlu.edu.cn

April 17, 2024

Navigation icons: back, forward, search, etc.

复习

- 系统模型的变换:

传递函数模型——零极点模型 `tf2zp`

零极点模型——传递函数模型 `zp2tf`

传递函数模型——状态空间模型 `tf2ss`

状态空间模型——传递函数模型 `ss2tf`

- 生成状态空间表达的系统:

`ss(A,B,C,D)`

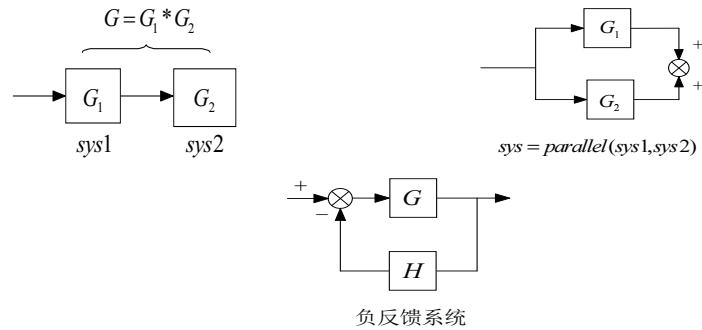
- 生成传递函数表达的系统:

`tf(num,den)`

Navigation icons: back, forward, search, etc.

复习

- 模型变换:



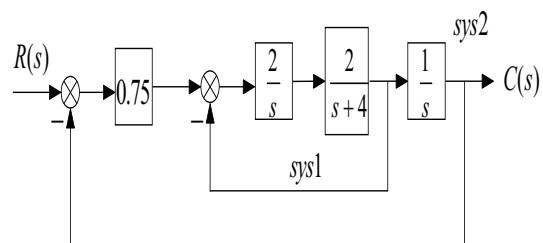
- 阶跃响应:

`step(num,den)` `[y,t]=step(num,den)`

Navigation icons: back, forward, search, etc.

复习

求下列系统的单位阶跃响应:



Navigation icons: back, forward, search, etc.

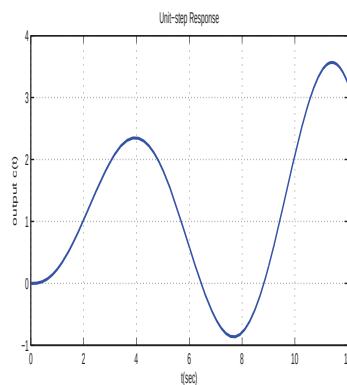
复习

程序：

```
>> t=0:0.01:12;  
>> num0=[4]; den0=[1 4 0]; sys0=tf(num0,den0);  
>> sys1=feedback(sys0,1);  
>> num2=[1]; den2=[1 0]; sys2=tf(num2,den2);  
>> sys3=series(0.75*sys1, sys2);  
>> sys=feedback(sys3,1);  
>> [c,t]=step(sys,t);  
>> plot(t,c)  
>> grid  
>> title('Unit-step Response')  
>> xlabel('t(sec)')  
>> ylabel('Output c(t)')
```

Navigation icons: back, forward, search, etc.

仿真图：



Navigation icons: back, forward, search, etc.

复习

例: 对由 $\frac{Y(s)}{R(s)} = \frac{6}{s^3+6s^2+11s+6}$

定义的系统, 试寻求一个形如 $\frac{Y(s)}{R(s)} = \frac{w_n^2}{s^2+2\xi w_n s+w_n^2}$

的二阶系统, 使它非常近似于给定三阶系统的单位阶跃响应.

复习

例: 对由 $\frac{Y(s)}{R(s)} = \frac{6}{s^3+6s^2+11s+6}$

定义的系统, 试寻求一个形如 $\frac{Y(s)}{R(s)} = \frac{w_n^2}{s^2+2\xi w_n s+w_n^2}$

的二阶系统, 使它非常近似于给定三阶系统的单位阶跃响应.

求解需要注意的问题:

- 搜索区域选定为 $0.65 \leq \xi \leq 1.00$, $0.5 \leq w_n \leq 4$.
- 判断是否近似, 在任一时刻 t , $[y_1(t) - y_2(t)]^2 \leq 0.05$.
- 输出在搜索区域里所有满足条件的向量 (ξ, w_n) , 搜索步长选定为 0.05.

程序：

```
clc
clear
t=1:0.01:6;
k=1;
num1=[6];den1=[1 6 11 6];sys1=tf(num1,den1);
[y1,t]=step(sys1,t);

for p=1:8
    zeta(p)=0.6+0.05*p;
    for q=1:8;
        w(q)=0.5*q;
        num2=[w(q)^2];den2=[1 2*w(q)*zeta(p) w(q)^2];
        sys2=tf(num2,den2);
        [y2,t]=step(sys2,t);

        error_square=(y1-y2).^2;
        emax=max(error_square);

        if emax<0.05;
            solution(k,:)=[k,zeta(p),w(q),emax];
            k=k+1;
        end
    end
end
end
```

Navigation icons: back, forward, search, etc.

复习

用 Matlab 求上升时间, 峰值时间, 最大超调量和调整时间:

eg: $G(s) = \frac{25}{s^2 + 6s + 25}$

Navigation icons: back, forward, search, etc.

程序:

```
num=[25];
den=[1 6 25];
t=0:0.005:5;
y=step(num,den,t)
% 上升时间
r=1;
while y(r)<1.0000
    r=r+1;
end
rise_time=(r-1)*0.005    %0.5550
%最大峰值时间
[ymax,tp]=max(y);
peak_time=(tp-1)*0.005    %0.7850
%超调量
max_overshoot=ymax-1    %0.0948
%调整时间
s=1001;
while y(s)>0.98 & y(s)<1.02 %1.1850
    s=s-1;
end
setting_time=(s-1)*0.005
```

Navigation icons: back, forward, search, etc.

冲激响应

- `impulse(num,den)`, `impulse(sys)`, `impulse(A,B,C,D)`
- `impulse(num,den,t)`, `impulse(sys,t)`
- `y=impulse(num,den)`, `y=impulse(sys)`
- `[y,t]=impulse(num,den)`
- `[y,t,x]=impulse(num,den)`

Navigation icons: back, forward, search, etc.

冲激响应

eg: 求系统 $G(s) = \frac{1}{s^2+0.2s+1}$ 的单位冲激响应.

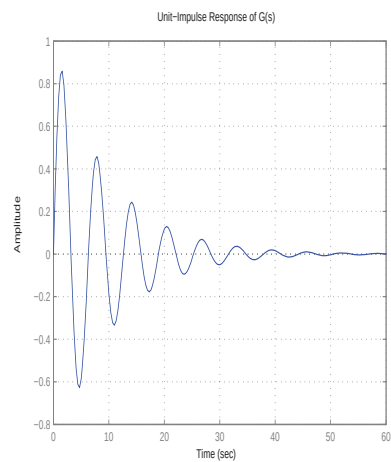
```
num=[1];  
den=[1 0.2 1];  
sys=tf(num,den);  
impulse(sys)      %impulse(num,den)  
grid  
title('Unit-Impulse Response of G(s)')
```

Navigation icons: back, forward, search, etc.

冲激响应

eg: 求系统 $G(s) = \frac{1}{s^2+0.2s+1}$ 的单位冲激响应.

```
num=[1];  
den=[1 0.2 1];  
sys=tf(num,den);  
impulse(sys)      %impulse(num,den)  
grid  
title('Unit-Impulse Response of G(s)')
```



Navigation icons: back, forward, search, etc.

任意指定输入的响应

`lsim(num,den,u,t)` `lsim(A,B,C,D,u,t)` `lsim(sys,u,t)`
`y=lsim(num,den,u,t)` `y=lsim(A,B,C,D,u,t)` `y=lsim(sys,u,t)`
`[y,t]=lsim(sys,u,t)`

- 时不变系统, 初始条件为零时, 对任意输入的响应.
- 可以用于单位斜坡响应 $u(t) = t$.
- 如果 $t = 0 : \Delta t : T$, 则从 $t=0$ 开始到 $t=T$ 结束的区间内每隔 Δt 秒计算一次响应.
- 不带左端参数的指令 `lsim(num, den, u, t)` 执行后直接绘制图形
- `y = lsim(num, den, u, t)` 会返回输出响应, 矩阵 `y` 的各列是输出, 它的行数等于 `t` 的长度. 但它不绘制图形, 绘制图形必须用命令 `plot(t, y)`.

单位斜坡响应

- 如果初始状态不为零, 那么命令 `lsim(sys, u, t, x0)` 会生成系统对输入 `u` 和初始条件 `x0` 的响应; 因为要明确系统的状态, 为了避免出错, 最好选择系统的状态空间模型
- 在调用 `lsim(sys, u, t)` 时, 系统初态 `x0` 是一个可选项
- `lsim(sys1, sys2, ..., u, t)` 可以在同一幅图中绘制多个系统、在相同输入下的响应.

eg: 求系统 $G(s) = \frac{1}{s^2+s+1}$ 的单位斜坡响应

单位斜坡响应

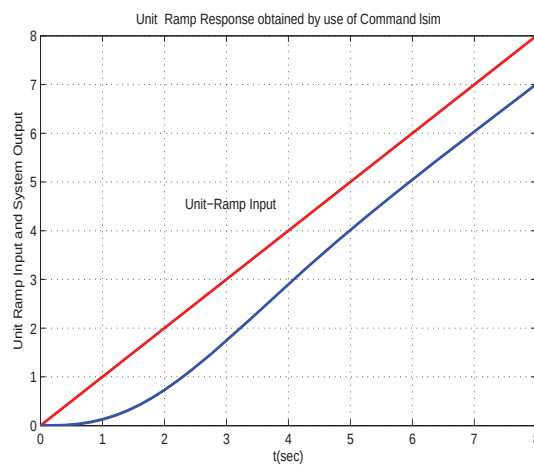
程序：

```
>> num=[1]; den=[1 1 1];  
>> t=0:0.1:8;  
>> r=t;  
>> y=lsim(num, den, r, t)  
>> plot(t, r, '-.', t, y, '0')  
>> grid  
>> xlabel('t(sec)')  
>> ylabel('Unit-Ramp Input and System Output')  
>> title('Unit-Ramp Response obtained by Use of Command lsim')  
>> text(2.1, 4.65, 'Unit-Ramp Input')
```

Navigation icons: back, forward, search, etc.

单位斜坡响应

仿真图：



Navigation icons: back, forward, search, etc.

单位斜坡响应

eg: 已知单位负反馈系统的开环传递函数为

$$\frac{C(s)}{R(s)} = \frac{s+10}{s^3+6s^2+9s+10}$$

(1) 试用 `lsim` 求闭环系统的单位斜坡响应;

(2) 在输入为 $r_1 = e^{-0.5t}$ 时的响应.

Navigation icons: back, forward, search, etc.

单位斜坡响应

程序:

```
num=[1 10];
den=[1 6 9 10];
sys1=tf(num,den);
sys=feedback(sys1,[1]);

t=0:0.1:10;
r=t;
y=lsim(sys,r,t);

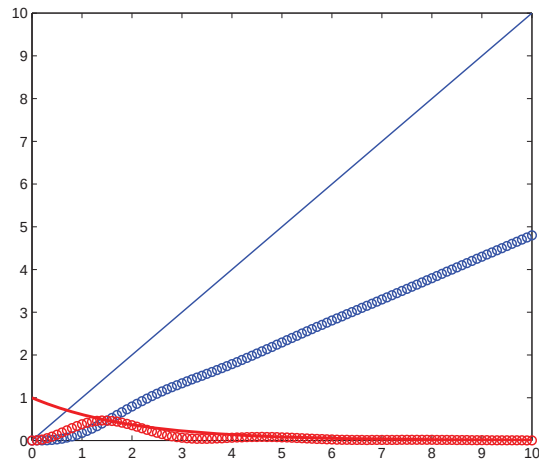
r1=exp(-0.5*t);
y1=lsim(sys,r1,t);

plot(t,r,'b-',t,y,'bo')
hold on
plot(t,r1,'r-',t,y1,'ro')
hold off
```

Navigation icons: back, forward, search, etc.

单位斜坡响应

程序：



零输入响应

(1) $[y,t,x]=\text{initial}(A,B,C,D,[\text{initial condition}],t)$

y: 输出 t: 向量 x: 状态

(2) $[y,t,x]=\text{initial}(\text{sys},[\text{initial condition}],t)$

sys: 状态空间描述的系统

零输入响应

eg: $\dot{x} = Ax + Bu$ $x(0) = x_0$

$$y = Cx + Du$$

系统参数: $A = \begin{pmatrix} 0 & 1 \\ -10 & -5 \end{pmatrix}$, $B = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$, $C = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$, $D = \begin{pmatrix} 0 & 0 \end{pmatrix}$,

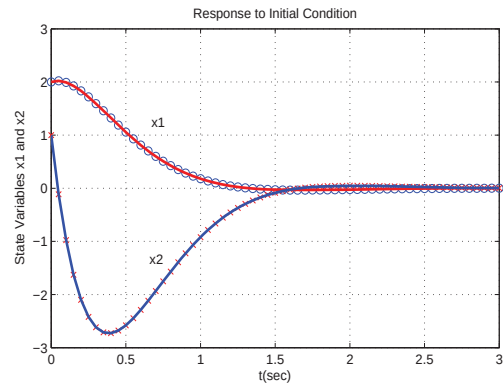
初值 $x_0 = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$.

零输入响应

程序:

```
>> t=0:0.05:3;  
>> A=[0 1;10 -5]; B=[0;0]; C=[1 0;0 1]; D=[0;0];  
>> [y,x]=initial(A,B,C,D,[2;1],t);  
>> plot(t,y(:,1),'o',t,y(:,2),'x');  
>> grid;  
>> title('Respond to Initial Condition');  
>> xlabel('t(sec)');  
>> ylabel('State Variable x1 and x2');  
>> gtex('x1');  
>> gtex('x2');
```

零输入响应



Isim

(2) 另一个函数 `Isim` 的非零初态下的响应 (也适用于状态空间描述):

$$[y, t, x] = \text{Isim}(\text{sys}, r, t, x_0)$$

其中 r 为输入.

练习

已知系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$
$$y = [1 \quad 0] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [0]u$$

初值为 $x_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

求在单位阶跃输入下的响应, 画图区间 $[0,10]$, 即 $t \in [0, 10]$

Navigation icons: back, forward, search, etc.

练习

已知系统

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u$$
$$y = [1 \quad 0] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + [0]u$$

初值为 $x_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

求在单位阶跃输入下的响应, 画图区间 $[0,10]$, 即 $t \in [0, 10]$

方法 1: initial + step

方法 2: lsim

Navigation icons: back, forward, search, etc.

控制系统数字仿真

于 树友

Jilin University
shuyou@jlu.edu.cn

April 22, 2024

Navigation icons: back, forward, search, etc.

学习过的命令/函数

- ① 求闭环系统的特征根 `r=roots(P)`
- ② 求部分分式展开 `residue`
- ③ 模型变换
`tf2zp`, `zp2tf`, `tf2ss`, `ss2tf`
- ④ 模型运算
`series(G1,G2)`, `parallel(G1,G2)`, `feedback(G,H)`
- ⑤ 阶跃响应
`step(num,den)`
`[y,t,x]=step(num,den)`
- ⑥ 冲击响应
`impulse(num,den)`
`[y,t,x]=impuse(num,den)` % 返回系统的输出, 时间向量, 状态, 调节时间

Navigation icons: back, forward, search, etc.

学习过的命令/函数

- 对任意输入的响应

`lsim(num,den,u,t,x0)`

- 对任意初始条件的响应

`[y, t, x] = initial(A, B, C, D, [initial condition], t)`

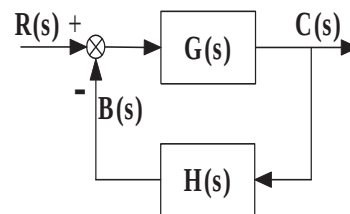
- 上升时间, 峰值时间, 超调量, 调节时间

Navigation icons: back, forward, search, etc.

根轨迹分析

已知系统的闭环传递函数为

$$\frac{C(s)}{R(s)} = \frac{G(s)}{1 + G(s)H(s)}$$



- 根轨迹: 系统的某一参数由 $0 \rightarrow \infty$ 变化时, 闭环极点 λ (特征方程的根) 在 s 平面相应变化所描绘出来的轨迹。
- 根轨迹里的根, 指的就是特征方程的根。

Navigation icons: back, forward, search, etc.

根轨迹分析

闭环系统的特征方程为

$$1 + G(s)H(s) = 0 \quad (*)$$

设 $G(s)H(s) = \frac{k(s+z_1)(s+z_2)\dots(s+z_m)}{(s+p_1)(s+p_2)\dots(s+p_n)}$ % 闭环系统的开环传递函数

则系统的特征方程可以写为:

$$1 + \frac{k(s+z_1)(s+z_2)\dots(s+z_m)}{(s+p_1)(s+p_2)\dots(s+p_n)} = 0$$

可以把 (*) 式改写成:

$$G(s)H(s) = -1$$

(1) 幅角条件 $\angle GH = \pm\pi(2k+1) \quad (k=0,1,2,\dots)$

(2) 幅值条件 $|GH| = 1$

- 同时满足幅角条件和幅值条件的 s 值就是特征方程的根, 或者说闭环极点; 满足幅角条件和幅值条件的点在复平面上的轨迹是根轨迹。

根轨迹分析

用来绘制根轨迹的 Matlab 命令

$$rlocus(num, den)$$

- 该命令在屏幕上绘制根轨迹图
- 对以状态空间形式定义的系统, $rlocus(A,B,C,D)$ 按照自动确定的增益向量来绘制系统的根轨迹
- 命令 $rlocus(num,den,k)$ 和 $rlocus(A,B,C,D,k)$ 按照用户确定的增益向量来绘制该系统的根轨迹

根轨迹分析

- 如果调用命令时带有左端参数，如

$[r, k] = rlocus(num, den)$

$[r, k] = rlocus(num, den, k)$

$[r, k] = rlocus(A, B, C, D)$

$[r, k] = rlocus(A, B, C, D, k)$

$[r, k] = rlocus(sys)$

- r 的行数等于 k 的长度，列数为 n ，这里 n 是分母多项式的系数
- r 包含根的位置。矩阵 r 的每一行对应于向量 k 中的一个增益
- 绘图命令 $plot(r, '-')$ 可以直接绘制根轨迹。
此时 Matlab 采用了 $plot$ 命令的自动坐标轴刻度特性

Navigation icons: back, forward, search, etc.

根轨迹分析

- 由于增益向量是自动确定的，所以

$$G(s)H(s) = \frac{k(s+1)}{s(s+2)(s+3)} \quad G(s)H(s) = \frac{10k(s+1)}{s(s+2)(s+3)}$$

$$G(s)H(s) = \frac{2000k(s+1)}{s(s+2)(s+3)}$$

的根轨迹图完全相同。

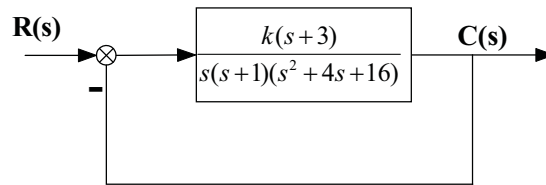
- 对应这三个系统而言，系统的 num 和 den 都是一样的：

$num = [0 \ 0 \ 1 \ 1]$

$den = [1 \ 5 \ 6 \ 0]$

Navigation icons: back, forward, search, etc.

根轨迹分析



用正方形的平面图形来绘制根轨迹, 使斜率为 1 的直线正好为 45°。
现将根轨迹的绘制区间选择为

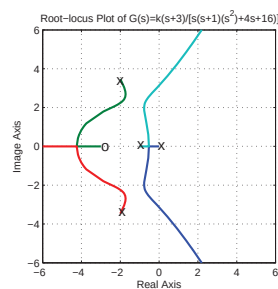
$$-6 \leq x \leq 6, \quad -6 \leq y \leq 6$$

其中 x 轴和 y 轴分别是实轴坐标和虚轴坐标。

根轨迹分析

程序

```
>> clf % clear current figure
>> num = [1 3];
>> den = conv([1 1 0], [1 4 16]);
>> r = rlocus(num, den);
>> plot(r, '-');
>> v = [-6 6 -6 6]; % 屏幕绘图区
>> axis(v);
>> axis('square');
>> grid
>> title('Root-locus Plot of G(s)=k(s+3)/[s(s+1)(s^2+4s+16)]')
>> xlabel('Real Axis'); ylabel('Image Axis')
>> gtext('o')
>> gtext('x') gtext('x') gtext('x') gtext('x')
```



根轨迹不同变化区间搜索步长不同

闭环系统开环传递函数如下，求对应的系统的根轨迹

$$G(s)H(s) = \frac{k}{s^4 + 1.1s^3 + 10.3s^2 + 5s}$$

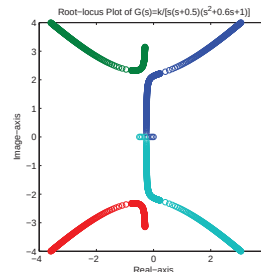
程序:

```
>> clf
>> num=[1]; den=[1 1.1 10.3 5 0];
>> k1=0:0.2:20; k2=20:0.1:30;
>> k3=30:5:1000; k=[k1 k2 k3];
>> r=rlocus(num,den,k); plot(r,'o')
>> v=[-4 4 -4 4];
>> axis(v); axis('square');

>> title('Root-locus Plot of G(s)=k/[s(s+0.5)(s^2+0.6s+1)]')
>> xlabel('Real-axis'); ylabel('Image-axis');
```

注:

利用根轨迹如何判断系统稳定性



根轨迹不同变化区间搜索步长不同

闭环系统开环传递函数如下，求对应的系统的根轨迹

$$G(s)H(s) = \frac{k}{s^4 + 1.1s^3 + 10.3s^2 + 5s}$$

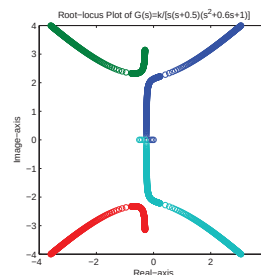
程序:

```
>> clf
>> num=[1]; den=[1 1.1 10.3 5 0];
>> k1=0:0.2:20; k2=20:0.1:30;
>> k3=30:5:1000; k=[k1 k2 k3];
>> r=rlocus(num,den,k); plot(r,'o')
>> v=[-4 4 -4 4];
>> axis(v); axis('square');

>> title('Root-locus Plot of G(s)=k/[s(s+0.5)(s^2+0.6s+1)]')
>> xlabel('Real-axis'); ylabel('Image-axis');
```

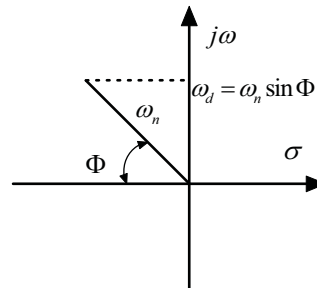
注:

利用根轨迹如何判断系统稳定性 ⇒ 根轨迹的图在复平面的左侧



带有极坐标网格线的根轨迹

- 等 ξ 轨迹和等 ω_n 轨迹
 $\xi = \cos \Phi$
- 等阻尼系数 ξ 的直线是穿过原点的射线; 等 ω_n 轨迹则是一系列的圆.
- 阻尼系数决定了极点的角位置, 而该极点与原点的距离则由无阻尼自然频率 ω_n 确定.



带有极坐标网格线的根轨迹

- 命令 `sgrid` 将等阻尼系数 ($\xi \in [0, 1]$, 步长为 0.1) 的直线轨迹和等 ω_n 的圆轨迹重叠绘在根轨迹图上.
- 如果希望得到特定 ξ 的直线轨迹 (如 $\xi = 0.5$ 的直线和 $\xi = 0.707$ 的直线) 及特定的 ω_n 的圆轨迹 (如 $\omega_n = 0.5$, $\omega_n = 1$, $\omega_n = 2$ 的圆), 可以采用如下命令

`sgrid([0.5 0.707],[0.5 1 2])`

- 如果要删去全部等 ξ 直线或全部等 ω_n 圆轨迹, 就可以在命令 `sgrid` 的参数中采用空括号 `[]`.
eg: `sgrid(0.5,[])` % 只在根轨迹图上重叠与 $\xi = 0.5$ 相应的等阻尼系数直线, 而不需要等 ω_n 图

带有极坐标网格线的根轨迹

例题：

已知单位负反馈系统具有如下前向通道传递函数

$$G(s) = \frac{s+2}{s^3 + 9s^2 + 8s}$$

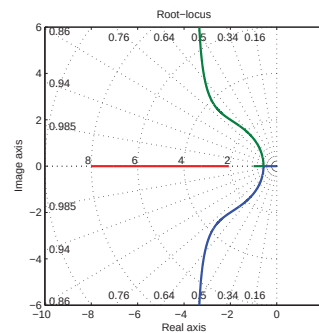
使用 Matlab 绘制根轨迹图，并在 s 平面上重叠绘制等 ξ 直线根轨迹和等 ω_n 圆轨迹。

Navigation icons: back, forward, search, etc.

带有极坐标网格线的根轨迹

● 程序：

```
num=[1 2];  
den=[1 9 8 0];  
k=0:0.02:200;  
r=rlocus(num,den,k);  
plot(r,'-'); v=[-10 2 -6 6];  
axis(v)  
axis('square')  
sgrid  
title('Root-locus')  
xlabel('Real axis')  
ylabel('Image axis')
```



Navigation icons: back, forward, search, etc.

求根轨迹上任一点的增益值 k

- 求根轨迹上任一点的增益值 k

$$[k, r] = rlocfind(num, den)$$

或者

$$[k, r] = rlocfind(A, B, C, D)$$

- 命令 `rlocfind` 必须在 `rlocus` 之后调用, 并且将可转移的 x-y 坐标覆盖在屏幕上.
- 使用鼠标将 x-y 坐标系的原点置于根轨迹上希望的点且按下鼠标左键, Matlab 就会显示该点的坐标.
- 该点的增益值与该点的增益值对应的闭环极点.

Navigation icons: back, forward, search, etc.

求根轨迹上任一点的增益值 k

例题:

已知单位负反馈系统具有如下前向通道传递函数

$$G(s) = \frac{K}{s(s^2 + 4s + 5)}$$

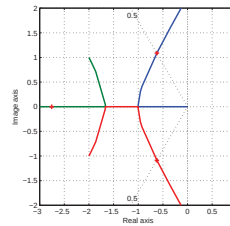
使用 Matlab 绘制根轨迹图, 确定阻尼系数为 0.5 的闭环极点, 并求出该点所对应的增益值 k.

Navigation icons: back, forward, search, etc.

求根轨迹上任一点的增益值 k

- 程序:

```
num=[1];
den=[1 4 5 0];
r=rlocus(num,den);
plot(r,'-');
v=[-3 1 -2 2];
axis(v); axis('square')
grid
sgrid(0.5,[ ]);
[K,r]=rlocfind(num,den)
xlabel('Real axis')
ylabel('Image axis')
```



```
selected_point = -0.6153 + 1.0854i
k = 4.3328
r = -2.7563
    -0.6219 + 1.0887i
    -0.6219 - 1.0887i
```

说明

该方法的闭环极点与采用解析法计算的位置稍有差别,其原因在于无法将 x-y 轴的点精确定位到根轨迹分支和 $\xi = 0.5$ 直线的交点上.

条件稳定系统的根轨迹

条件稳定系统

- 系统只是在有限的 K 值范围内才是稳定的;
- 如果将 K 取成与不稳定运行相对应的值, 该系统就可能遭到破坏, 或者由于可能存在的饱和而成为非线性系统;
- 条件稳定系统不是人们所希望的系统;
- 附加某个适合的校正网络, 可以消除条件稳定性.

条件稳定系统的根轨迹

例题：

对于具有纵向模式自动驾驶仪的飞机，它的开环传递函数的简化形式是：

$$G(s)H(s) = \frac{k(s+1)}{s(s-1)(s^2+4s+16)}$$

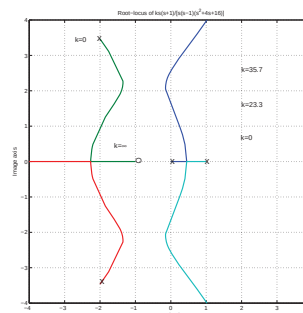
绘制系统的根轨迹，并求出使系统稳定所需的增益 k 的范围。

Navigation icons: back, forward, search, etc.

条件稳定系统的根轨迹

● 程序：

```
num=[1 1];  
den=conv([1 -1 0],[1 4 16])  
r=rlocus(num,den);  
plot(r,'-'); v=[-4 4 -4 4];  
axis(v); axis('square'); grid  
title('Root-locus of  
k(s+1)/[s(s-1)(s^2+4s+16)]')  
xlabel('Real axis')  
ylabel('Image axis')
```



Navigation icons: back, forward, search, etc.

条件稳定系统的根轨迹

根轨迹穿过虚轴的值可以由劳斯稳定判据来确定

特征方程为

$$s^4 + 3s^3 + 12s^2 + (k - 16)s + k = 0$$

因而劳斯阵列为：

s^4	1	12	k
s^3	3	$k-16$	
s^2	$\frac{52-k}{3}$	k	
s^1	$\frac{-k^2+59k-52 \cdot 16}{52-k}$	0	
s^0	k		

① 令 s^1 第一列交点处的项为零，
所求得的 k 值是

$$k = 35.7 \text{ 和 } k = 23.3$$

② 和虚轴的交点可以由解 s^2 行构成的辅助方程求取，即

$$\frac{52-k}{3}s^2 + k = 0$$

求解为：

$$s = \pm j2.56, \quad k = 35.7$$

$$s = \pm j1.56, \quad k = 23.3$$

③ 保证稳定性所需的 k 值范围为
 $23.3 < k < 35.7$

条件稳定系统的根轨迹

● 搜索求取使得系统稳定的增益 k 的范围：

```
k=0:0.01:50;
```

```
num=[1 1];
```

```
den=conv([1 -1 0],[1 4 16])
```

```
[r,k]=rlocus(num,den,k);
```

```
kn=[];
```

```
for j=1:1:5001
```

```
    if(all(real(r(j))< 0))
```

% 判断根是否在左半平面

```
        kn=[[kn]; j];
```

```
        sk=0.01*(kn-1);
```

```
    end
```

```
end
```

```
r1=min(sk)
```

```
r2=max(sk)
```

非最小相位系统的根轨迹

最小相位系统

- 如果系统的所有极点和所有零点都位于左半 s 平面, 该系统就称为最小相位系统.

非最小相位系统

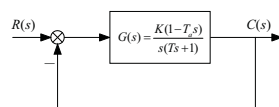
- 如果系统至少有一个零点或者极点位于右半 s 平面, 该系统就称为非最小相位系统.

非最小相位系统的根轨迹

系统框图如下图所示, 其中

$$G(s) = \frac{K(1-T_as)}{s(Ts+1)}, \quad (T_a > 0), \quad H(s) = 1$$

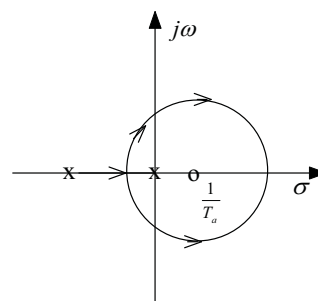
这是一个非最小相位系统, 因为它有一个右半 s 平面的零点.



对这个系统, 幅角条件成为

$$\begin{aligned} \angle G(s) &= \angle \frac{k(T_as-1)}{s(Ts+1)} \\ &= \angle \frac{k(T_as-1)}{s(Ts+1)} + 180^\circ \\ &= \pm 180^\circ (2k+1) \quad k=0,1,2,\dots \end{aligned}$$

$$\text{或者} \quad \angle \frac{k(T_as-1)}{s(Ts+1)} = 0^\circ$$

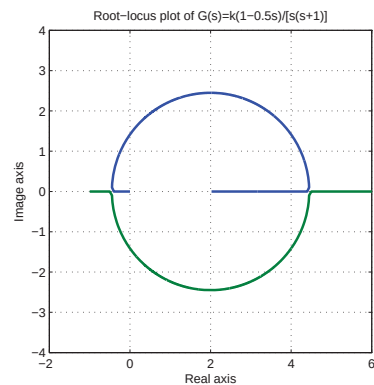


由根轨迹图可知, 如果增益 k 小于 $\frac{1}{T_a}$, 该系统是稳定的.

非最小相位系统的根轨迹

程序：

```
num=[-0.5 1]; den=[1 1 0];  
k1=0:0.01:30;  
k2=30:1:100;  
k3=100:5:500;  
k=[k1 k2 k3];  
r=rlocus(num,den,k);  
plot(r); v=[-2 6 -4 4];  
axis(v); axis('square'); grid  
title('Root-locus plot of  
G(s)=k(1-0.5s)/[s(s+1)]')  
xlabel('Real axis')  
ylabel('Image axis')
```



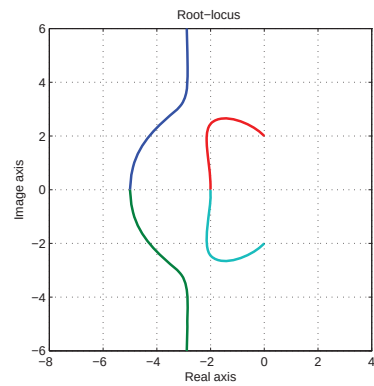
设单位反馈系统有如下开环传递函数：

$$G(s) = \frac{k(s+2)^2}{(s^2+4)(s+5)^2}$$

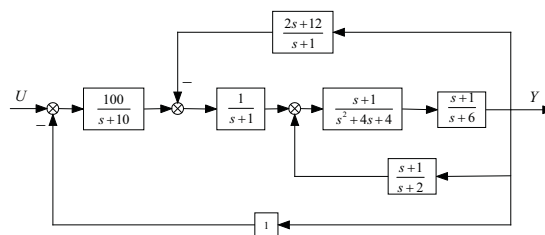
试用 Matlab 绘制该系统的根轨迹图，并判断该系统稳定性。

程序:

```
num=[1 4 4];
den=conv([1 0 4],[1 10 25]);
r=rlocus(num,den);
plot(r);
v=[-8 4 -6 6];
axis(v);
axis('square');
hold % Hold current graph
plot(r,'-')
grid
title('Root-locus ')
xlabel('Real axis')
ylabel('Image axis')
```



课后练习



对上图所示的多环控制系统, 试完成:

- (1) 系统的传递函数模型;
- (2) 零初始状态下, 求该系统的单位阶跃响应;
- (3) 分析系统稳定性; 判断是否达到稳态;
- (4) 求 $\sin(0.1t)$ 的响应.

控制系统数字仿真

于 树友

Jilin University
shuyou@jlu.edu.cn

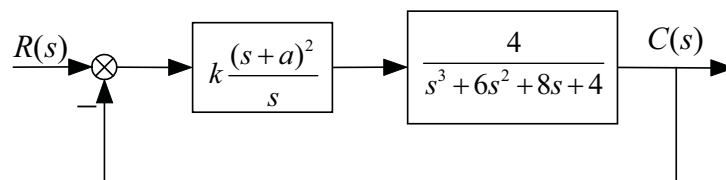
April 24, 2024

Navigation icons: back, forward, search, etc.

设计举例

设计举例

设计如下控制器, 使闭环系统为欠阻尼系统, 其单位阶跃响应的最大超调量小于 15% 且大于 10% .



(1) 特殊结构的 PID 控制器:

$$k \frac{(s+a)^2}{s} = k \left(2a + \frac{a^2}{s} + s \right)$$

Navigation icons: back, forward, search, etc.

设计举例

(2) 该系统的闭环传递函数:

$$\frac{C(s)}{R(s)} = \frac{4k(s+a)^2}{s(s^3 + 6s^2 + 8s + 4) + 4k(s+a)^2}$$

单位阶跃响应的稳态值:

$$\lim_{s \rightarrow 0} sC(s) = \lim_{s \rightarrow 0} s \cdot \frac{C(s)}{R(s)} \cdot \frac{1}{s} = \lim_{s \rightarrow 0} \frac{C(s)}{R(s)} = 1$$

因而, 根据设计要求只需:

$$1.10 < y_{max} < 1.15$$

Navigation icons: back, forward, search, etc.

设计举例

(3) 搜索 k 和 a 值, 使得系统满足要求.

增益 k 不应太大, 以避免需要过大的功率原件, 且影响系统的稳定性.

(4) 满足要求即可, 不必求出最佳的 k 和 a 值.

a) 从 k 和 a 值的最小端入手, 寻找到即停止.

b) 搜索区间定为:

$$3 \leq k \leq 5 \quad 0.1 \leq a \leq 3$$

c) 搜索步长越小越好, 但初始时需选大一些, 提高搜索效率.

Navigation icons: back, forward, search, etc.

设计举例

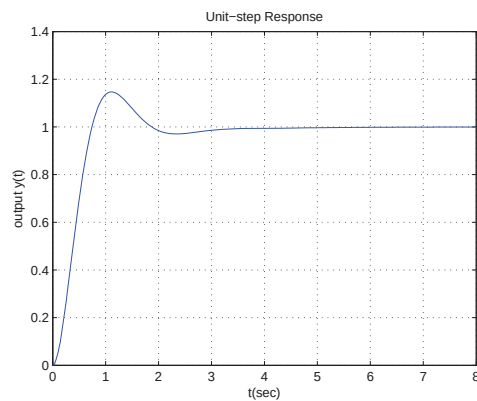
程序:

```
t = 0 : 0.01 : 8;
for k = 3 : 0.2 : 5
    for a = 0.1 : 0.1 : 3
        num1 = k * [1 2 * a a ^ 2];
        den1 = [10];
        tf1 = tf(num1, den1);
        num2 = [4];
        den2 = [1 6 8 4];
        tf2 = tf(num2, den2);
        tf3 = series(tf1, tf2);
        sys = feedback(tf3, 1);
        y = step(sys, t);
        m = max(y);
        if m < 1.15 & m > 1.10
            plot(t,y)
            grid
            title('Unit-step Response')
            xlabel('t(sec)')
            ylabel('output y(t)')
            solution=[k a m]
            break % Break the inner loop
        end
    end
end
end
end
```

设计举例

输出结果:

```
solution=
    3.0000    1.0000    1.1469
```



讨论

- 内置了循环嵌套，所以一定要设置跳出循环的指令，避免出错
- 目前只考虑了超调量，可以用多维搜索，考虑稳定性、稳态误差和上升时间、峰值时间.....

通过这种搜索的方式，实现系统的多目标控制，甚至是冲突目标的协调

- 这种方法在控制理论实践中仍然广泛采用.....

用 Matlab 绘制波特图

- 1 bode 命令可以计算连续线性定常系统的频域响应的幅角和相角
- 2 在 Matlab 中输入指令 bode (不带左端参数)，屏幕上就会画出波特图，图中的幅值单位为分贝 (dB)

bode(num,den)	bode(num,den, ω)
bode(sys)	bode(A,B,C,D, ω)

用 Matlab 绘制波特图

使用左端参数时, 例如采用

$$[mag, phase, \omega] = bode(num, den, \omega)$$

- ❶ 命令 `bode` 只将系统频率响应的数值赋予矩阵变量 `mag`, `phase` 和 ω , 而不在屏幕上绘制波特图.
- ❷ 矩阵 `mag` 和 `phase` 分别包含系统响应在指定频率点上算得的幅值和相角.
- ❸ 相角的单位为度; 幅值可以转化为分贝值

$$magdB = 20 * \log_{10}(mag)$$

Navigation icons: back, forward, search, etc.

用 Matlab 绘制波特图

- ❶ 带左端参数的 `bode` 命令的常用形式:

$$[mag, phase, \omega] = bode(num, den)$$

$$[mag, phase, \omega] = bode(num, den, \omega)$$

$$[mag, phase, \omega] = bode(sys)$$

$$[mag, phase, \omega] = bode(A, B, C, D, \omega)$$

- ❷ 在绘制波特图时, 如果需要采用用户指定的频率点, 那么 `bode` 命令必须像命令

$$bode(num, den, \omega) \text{ 或 } [mag, phase, \omega] = bode(num, den, \omega)$$

那样包含频率向量.

Navigation icons: back, forward, search, etc.

用 Matlab 绘制波特图

- ⑥ 如果指定频率范围, 可使用命令

`logspace(d1,d2)`: 在 10 倍频程 10^{d1} 和 10^{d2} 内生成一个包含 50 个点 (包括边缘点) 的, 呈对数均匀分布的向量.

`logspace(d1,d2,n)`: 在 10 倍频程 10^{d1} 和 10^{d2} 内呈对数均匀分布的 n 个点 (包括边缘点).

eg: 为了在频率 $0.1 \sim 100\text{rad/s}$ 之间生成 50 个点, 可以输入命令

$$\omega = \text{logspace}(-1, 2, 50)$$

用 Matlab 绘制波特图

例题:已知系统开环传递函数为

$$G(s) = \frac{9(s^2 + 0.2s + 1)}{s(s^2 + 1.2s + 9)}$$

绘制 $G(s)$ 的波特图, 要求:

- (1) $\omega \in 10^{-2} \sim 10^3$
- (2) 幅值在 $-50\text{dB} \sim 50\text{dB}$
- (3) 相角在 $-150^\circ \sim 150^\circ$ 之间

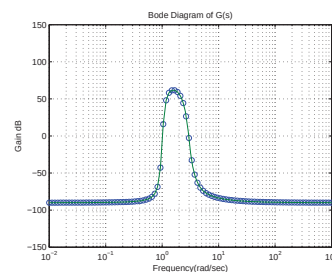
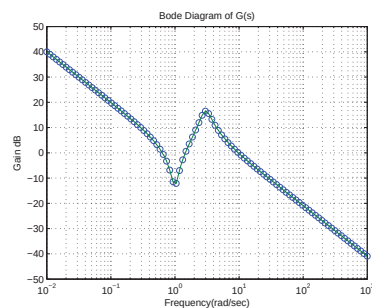
用 Matlab 绘制波特图

程序:

```
>> num=[9 1.8 9]; den=[1 1.2 9 0];  
>> w=logspace(-2,3,100); [mag,phase,w]=bode(num,den,w);  
>> figure(1)  
>> magdB=20*log10(mag);  
>> semilogx(w,magdB,'o',w,magdB,'-')  
>> axis([w(1),w(100),-50,50]); grid  
>> title('Bode Diagram of G(s)')  
>> xlabel('Frequency(rad/sec)'); ylabel('Gain dB')  
  
>> figure(2)  
>> semilogx(w,phase,'o',w,phase,'-')  
>> axis([w(1),w(100),-150,150]); grid  
>> title('Bode Diagram of G(s)')  
>> xlabel('Frequency(rad/sec)') ylabel('Phase deg')
```

用 Matlab 绘制波特图

程序运行结果:



用 Matlab 绘制波特图

已知系统:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -25 & -24 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ 25 \end{bmatrix} u$$
$$y = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 & x_2 \end{bmatrix}$$

使用命令 `bode(A,B,C,D)` 可以绘制系统的波特图.

用 Matlab 绘制奈奎斯特图

Nyquist 图是极坐标图

- $G(j\omega)$ 的极坐标图是当 ω 从零变化到无穷大时 ($0 \rightarrow \infty$), $G(j\omega)$ 的幅值和相角关系在极坐标系上的图形, 即 $\omega = 0 \rightarrow \infty$ 时, $|G(j\omega)|$ 和 $\angle G(j\omega)$ 形成的轨迹.
- 在极坐标图中, 正相角是从正实轴逆时针方向测量的相角; 负相角是从正实轴顺时针方向测量的相角.

Nyquist 稳定判据

- 开环传递函数没有 s 右半平面的极点: $G_k(j\omega)$ 逆时针不包围 $(-1, j0)$ 点;
- 开环传递函数有 P_k 个右半平面极点: $G_k(j\omega)$ 逆时针包围 $(-1, j0)$ 点 P_k 圈.

用 Matlab 绘制奈奎斯特图

Matlab命令:

`nyquist(num, den)`

- 如果在调用时不带左端参数, `nyquist` 就在屏幕上绘制奈式曲线;
- 与 `bode` 命令输入输出方式相同

`[re, im,  $\omega$ ] = nyquist(num, den,  $\omega$ )`

- 矩阵 `re` 和 `im` 分别为系统在由向量 ω 给定的频率上的频域响应的实部和虚部.
- 若要绘制奈式曲线, 可以使用 `plot`.
- 使用 `nyquist` 命令 (不带左端参数), 接下来再使用 `grid` 命令, 则不会产生 x-y 网格线;
如果需要使用网格线, 可以使用带左端参数的 `nyquist` 指令, 再使用 `grid` 命令来实现.

Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

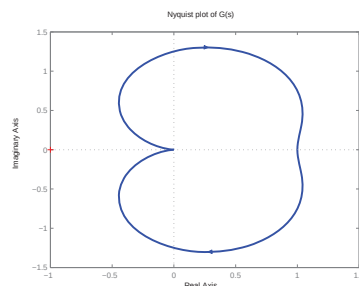
例题: 已知开环传递函数为:

$$G(s) = \frac{1}{s^2 + 0.8s + 1}$$

用 Matlab 绘制该系统的奈奎斯特图.

程序:

```
num = 1;  
den = [1 0.8 1];  
nyquist(num, den);  
title('Nyquist plot of G(s)')
```



Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

绘制如下传递函数的奈奎斯特曲线:

$$G(s) = \frac{1}{s(s+1)}$$

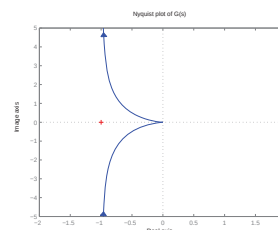
- Matlab 运算涉及 “Divided by zero” (被零除), 所绘制的传递函数的奈奎斯特曲线可能不正确;
- 可以通过指定的 `axis(v)` 来修正图形.

Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

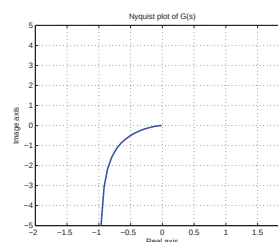
程序:

```
num=1; den=[1 1 0];  
nyquist(num,den)  
v=[-2 2 -5 5]; axis(v);  
title('Nyquist plot of G(s)')  
xlabel('Real axis')  
ylabel('Image axis')
```



若只需要频率为正的部分对应的曲线, 可对蓝色部分做如下修改:

```
w = 0.1 : 0.1 : 10  
[re, im, w] = nyquist(num, den, w)  
plot(re,im)  
v=[-2 2 -5 5];  
axis(v); grid
```



Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

绘制如下传递函数的奈奎斯特曲线：

$$G(s) = \frac{20(s^2 + s + 0.5)}{s(s + 1)(s + 10)}$$

在图中标明频率 $\omega = 0.2, 10 \text{ rad/s}$ 所对应的点，并且求解 $G(j\omega)$ 在这些频率点的幅值和相角。

Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

程序：

```
num=20*[1 1 0.5];
den=conv([1 1 0],[1 10]);
w1 = 0.1 : 0.1 : 10;
w2 = 10 : 2 : 100;
w3 = 100 : 10 : 500;
w = [w1 w2 w3];
ww = [0.2 20];
[re, im, w] = nyquist(num, den, w);
plot(re,im)
v=[-3 3 -5 1]; axis(v);
hold
[re1, im1, w1] =
nyquist(num, den, ww);
plot(re1,im1,'o')
text(1.1,-4.8,'w1=0.2')
text(0.77,-0.8,'20')
[mag phase w] =
bode(num, den, ww)
grid
title('Nyquist plot of G(s)')
xlabel('Real axis')
ylabel('Image axis')
```

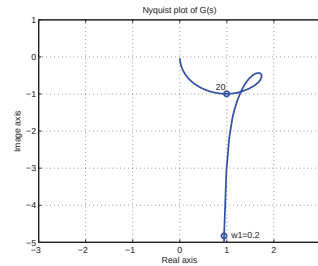
注：采用不同增量的子区间，根据曲线的密集程度设定

Navigation icons: back, forward, search, etc.

用 Matlab 绘制奈奎斯特图

输出结果：

```
w = 0.2000 20.0000
phase = -78.9571 -45.0285
mag = 4.9176 1.4072
```



用 Matlab 绘制奈奎斯特图

已知单位正反馈系统的开环传递函数

$$G(s) = \frac{s^2 + 4s + 6}{s^2 + 5s + 4}$$

绘制该传递函数的奈氏曲线。

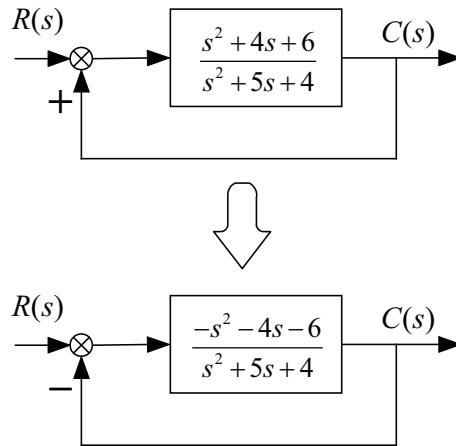
产生的问题

该系统为正反馈系统, 而奈氏稳定判据的应用对象为负反馈系统。

用 Matlab 绘制奈奎斯特图

解决思路:

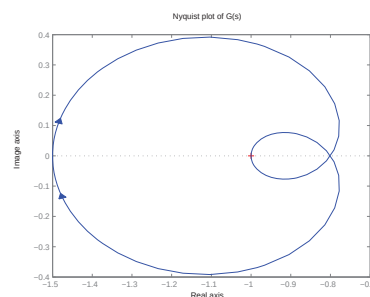
寻找等效的负反馈系统的开环传递函数



用 Matlab 绘制奈奎斯特图

程序:

```
num=[-1 -4 -6];  
den=[1 5 4];  
nyquist(num,den)  
v=[-1.5 -0.7 -0.4 0.4]; axis;  
title('Nyquist plot of G(s)')  
xlabel('Real axis')  
ylabel('Image axis')
```



注: 正反馈系统的奈氏图恰好是负反馈系统奈氏图关于虚轴对称的镜像

小结

① 画波特图

不带左端参数

$$bode(num, den, \omega)$$

直接画图, 图中幅值为分贝

带左端参数

$$[mag, phase, \omega] = bode(num, den, \omega)$$

不直接画图, 图中幅值只是增益, 需要用

$$magdB = 20 * \log_{10}(mag)$$

② $\logspace(d1, d2)$, $\logspace(d1, d2, n)$

③ 画奈氏曲线 nyquist

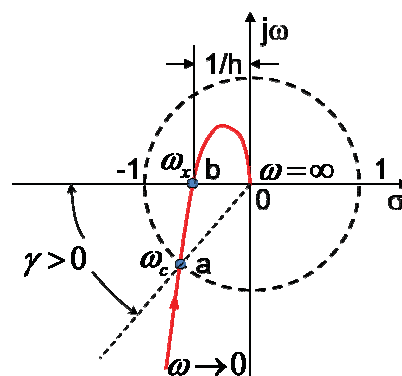
④ 正反馈系统的奈氏曲线

⇒ 需转化为等效的负反馈

系统的相对稳定性和稳定裕度

相角裕度:

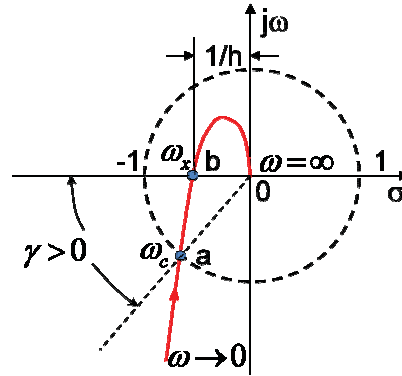
- $|G_k(j\omega)| = 1$ 或 $L(\omega_c) = 0$ 时, 图中点 a 到 $(-1, j0)$ 点之间的相角大小记为 γ , 称为系统的相角裕度.
- $\gamma = 180^\circ + \varphi(\omega_c)$



系统的相对稳定性和稳定裕度

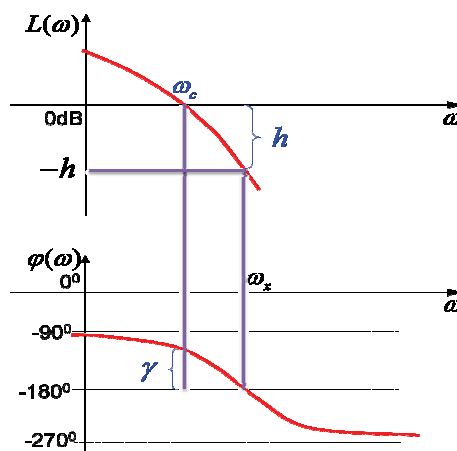
幅值裕度:

- 当 $\varphi(\omega_g) = -180^\circ$ 时, $|G_k(j\omega)|$ 与 $(-1, j0)$ 点之比, 称为幅值裕度.
- $kg = \frac{1}{|G_k(j\omega)|}$ 或 $kg(dB) = -20 \lg |G_k(j\omega)|$



工程中, 一般要求 $\gamma = 30^\circ \sim 60^\circ$, $kg(dB) = 6 \sim 20dB$

系统的相对稳定性和稳定裕度

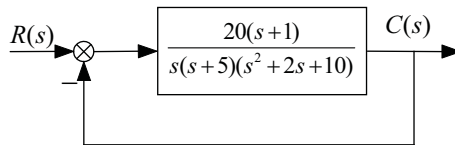


系统的相对稳定性和稳定裕度

Matlab 求增益裕度, 相角裕度, 相角穿越频率和增益穿越频率.

$$[G_m, P_m, w_{cp}, w_{cg}] = \text{margin}(\text{sys})$$

eg: 绘制 $G(s)$ 的波特图, 并求取增益裕度, 相角裕度, 相角穿越频率和增益穿越频率.



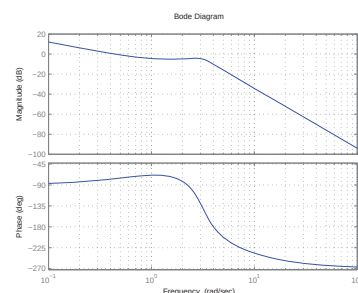
系统的相对稳定性和稳定裕度

程序:

```
>> num=[20 20];
>> den=conv([1 5 0],[1 2 10]);
>> sys=tf(num,den);
>> w=logspace(-1,2,100);
>> bode(sys,w)
>> grid
>> [Gm,Pm,wcp,wcg]=margin(sys);
>> GmdB=20*log10(Gm);
>> [GmdB,Pm,wcp,wcg]
```

输出结果:

3.1369 103.6573 4.0132 0.4426



系统的相对稳定性和稳定裕度

② 求谐振峰值, 谐振频率和带宽的方法

- 谐振峰值是闭环系统频率响应幅值的最大值;
- 谐振频率是产生最大幅值的频率点.

```
[mag, phase, w] = bode(num, den, w)
```

```
[Mp, k] = max(mag);
```

```
resonant_peak = 20 * log10(Mp);
```

```
resonant_frequency = w(k)
```

- 求取带宽

```
n=1;
```

```
while 20 * log10(mag(n)) >= -3
```

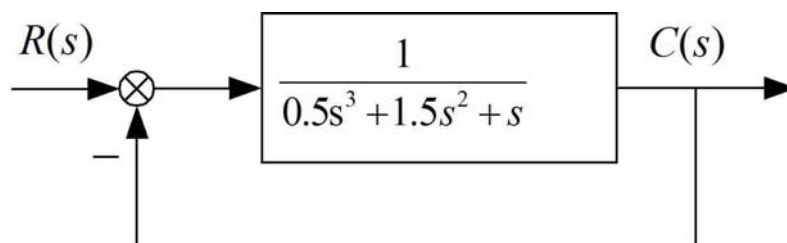
```
    n=n+1;
```

```
end
```

Navigation icons: back, forward, search, etc.

系统的相对稳定性和稳定裕度

已知系统的结构图



求谐振峰值, 谐振频率和带宽

Navigation icons: back, forward, search, etc.

系统的相对稳定性和稳定裕度

程序:

```
num=1; den=[0.5 1.5 1 0];
sys1=tf(num,den);
sys=feedback(sys1,1);
w=logspace(-1,1);
bode(sys,w), grid
[mag,phase,w]=bode(sys,w);
[Mp,k]=max(mag);
resonant_peak = 20 * log10(Mp)
resonant_frequency = w(k)
n=1;
while 20 * log10(mag(n)) >= -3
    n=n+1;
end
bandwidth = w(n)
```

输出结果:

resonant_peak = 5.2388

resonant_frequency = 0.7906

bandwidth = 1.2649

